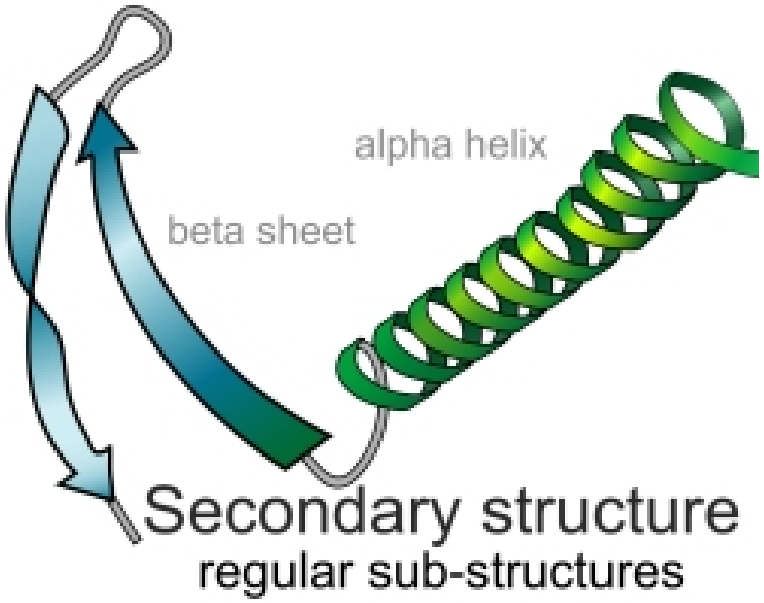
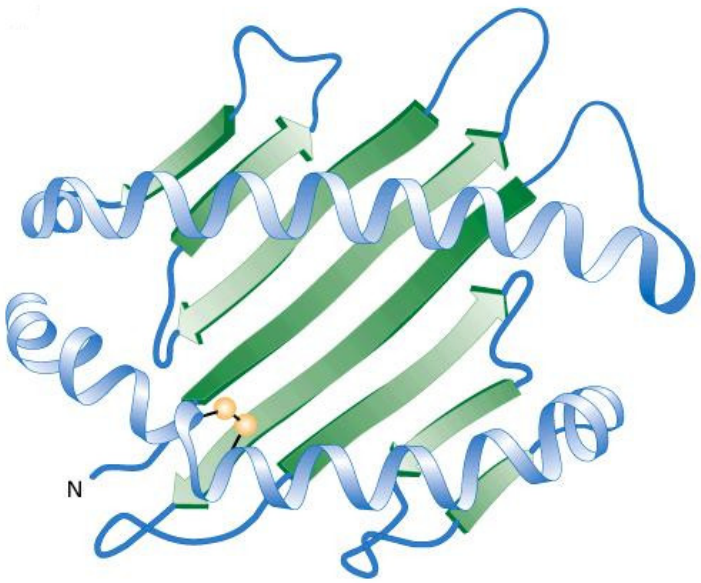
Lecture 9: Neural Networks for Biological Prediction

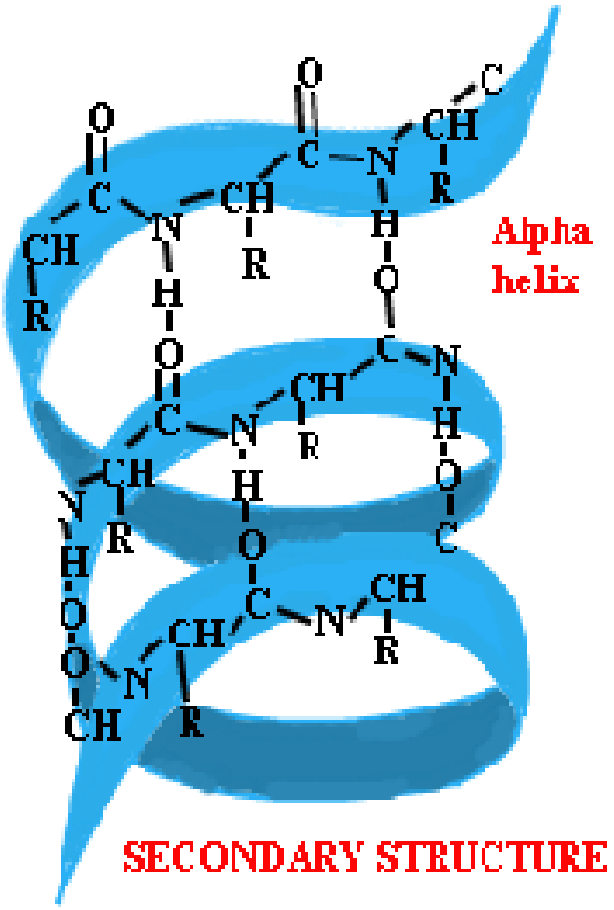




MLP, One-Hot Vectors, Softmax, Profile, Psi-Blast

Outline

1. **MLP**
2. Secondary Protein Structure
3. Designing MLPs



Neural Networks in Biology

- Neural networks (and support vector machines) are frequently used to make prediction in biology
- We are not going into detail about neural networks (many of you have done this in machine learning)
- However, how to use them in biology is quite subtle and is the main topic of this lecture
- We take as a case study the classic problem of the prediction of secondary structure of proteins
- Although, some of this is dated, the lessons are timeless

Neural Networks in Biology

- Neural networks (and support vector machines) are frequently used to make prediction in biology
- We are not going into detail about neural networks (many of you have done this in machine learning)
- However, how to use them in biology is quite subtle and is the main topic of this lecture
- We take as a case study the classic problem of the prediction of secondary structure of proteins
- Although, some of this is dated, the lessons are timeless

Neural Networks in Biology

- Neural networks (and support vector machines) are frequently used to make prediction in biology
- We are not going into detail about neural networks (many of you have done this in machine learning)
- However, how to use them in biology is quite subtle and is the main topic of this lecture
- We take as a case study the classic problem of the prediction of secondary structure of proteins
- Although, some of this is dated, the lessons are timeless

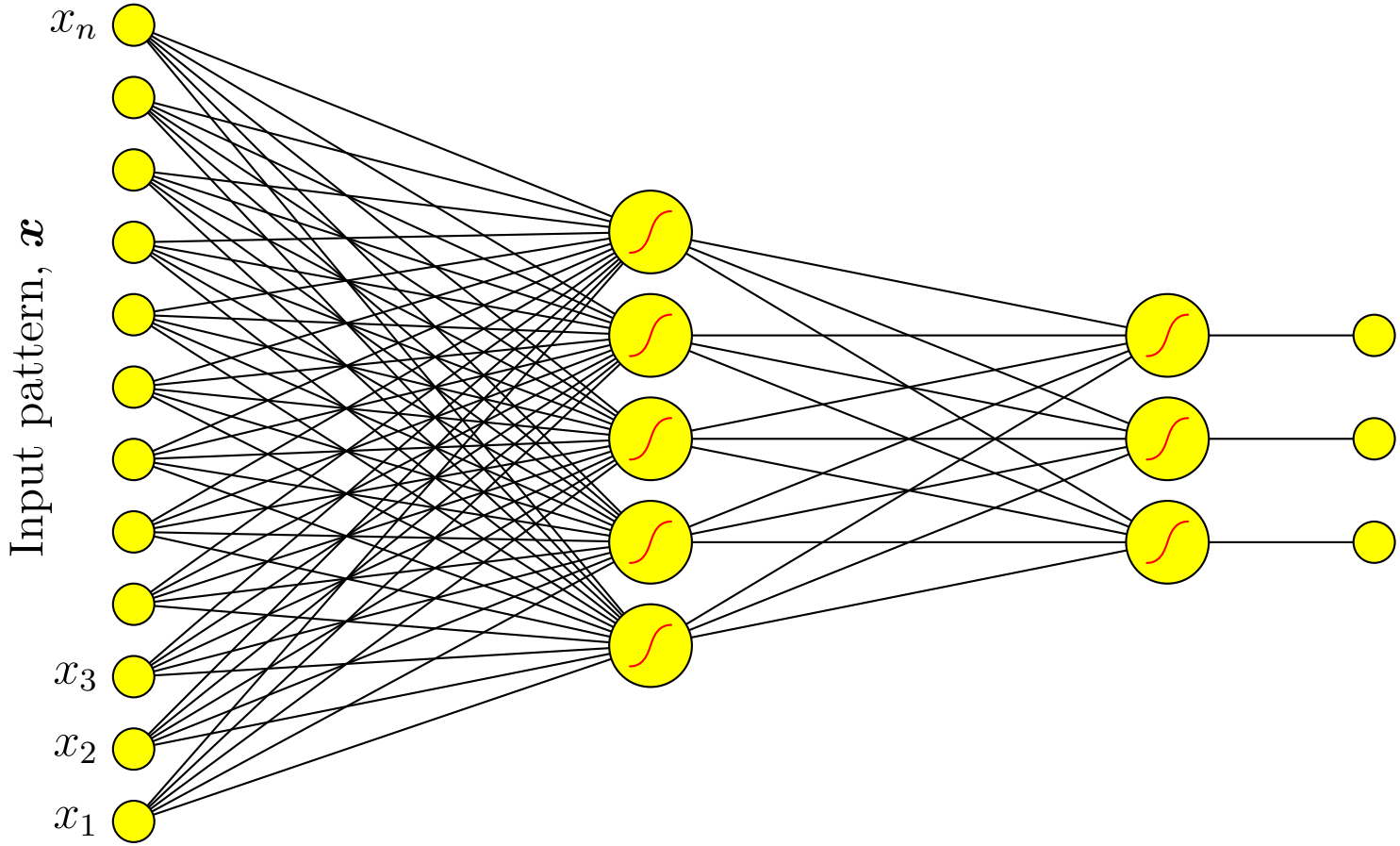
Neural Networks in Biology

- Neural networks (and support vector machines) are frequently used to make prediction in biology
- We are not going into detail about neural networks (many of you have done this in machine learning)
- However, how to use them in biology is quite subtle and is the main topic of this lecture
- We take as a case study the classic problem of the prediction of secondary structure of proteins
- Although, some of this is dated, the lessons are timeless

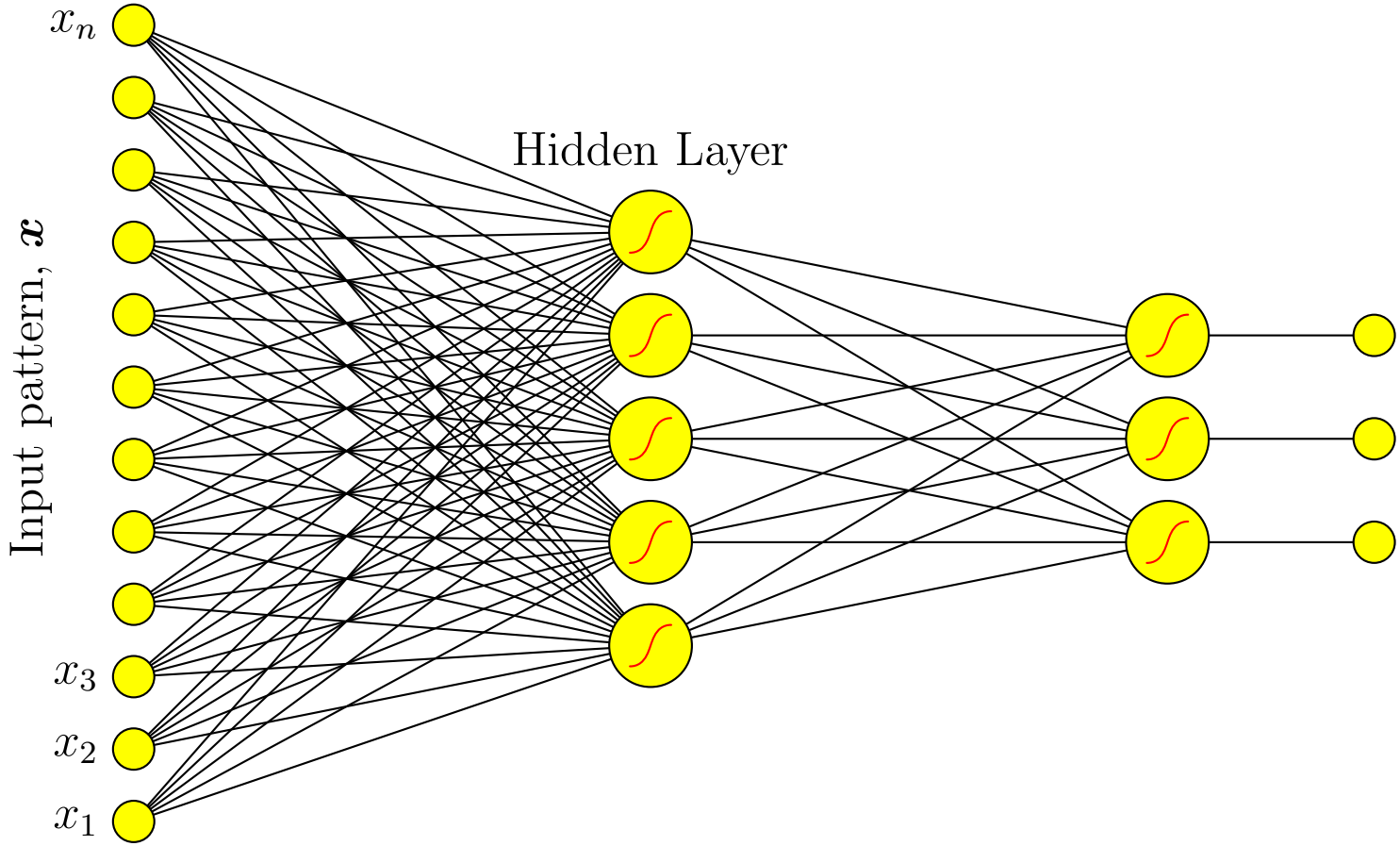
Neural Networks in Biology

- Neural networks (and support vector machines) are frequently used to make prediction in biology
- We are not going into detail about neural networks (many of you have done this in machine learning)
- However, how to use them in biology is quite subtle and is the main topic of this lecture
- We take as a case study the classic problem of the prediction of secondary structure of proteins
- Although, some of this is dated, the lessons are timeless

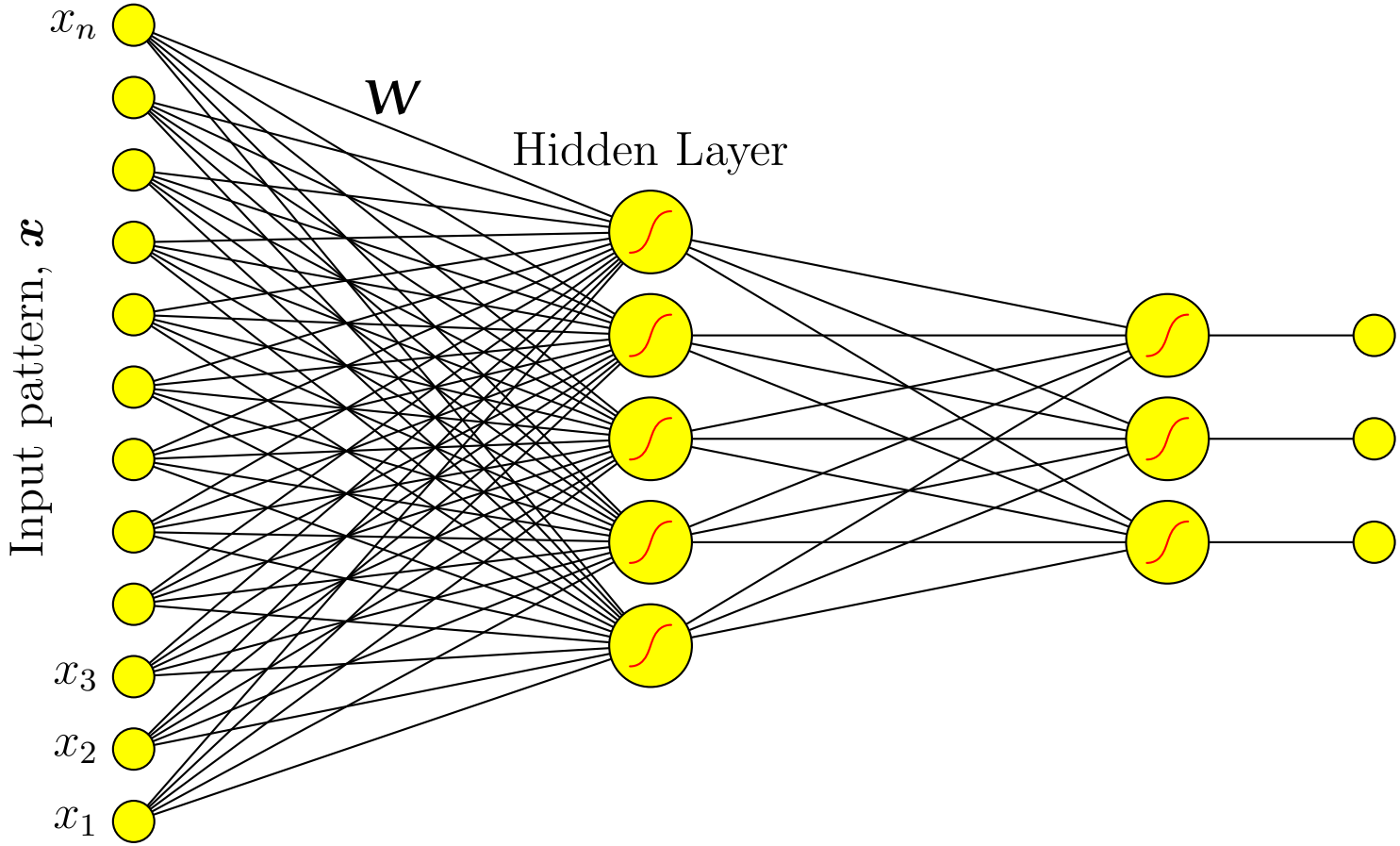
Multilayer Perceptron (MLP)



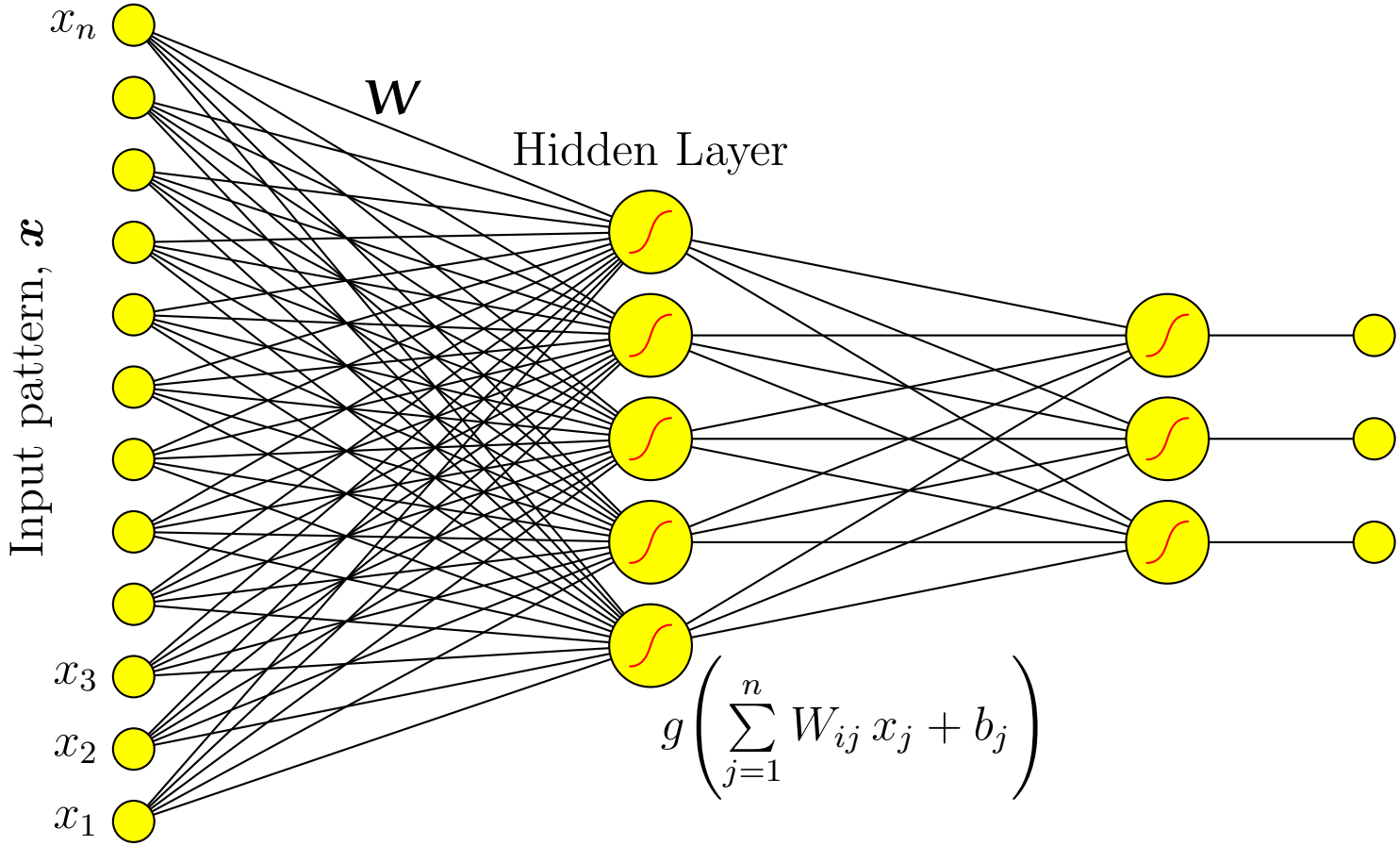
Multilayer Perceptron (MLP)



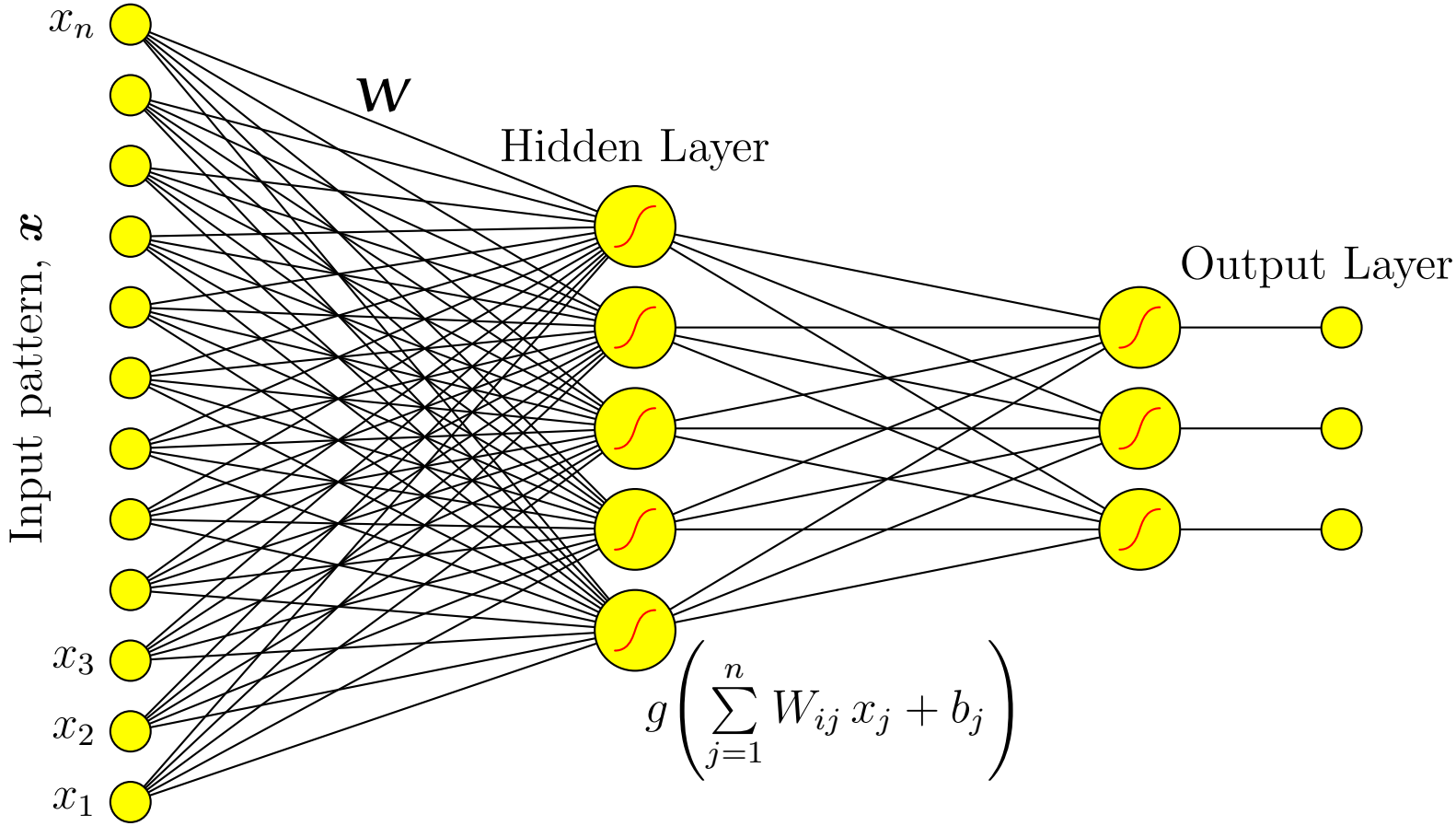
Multilayer Perceptron (MLP)



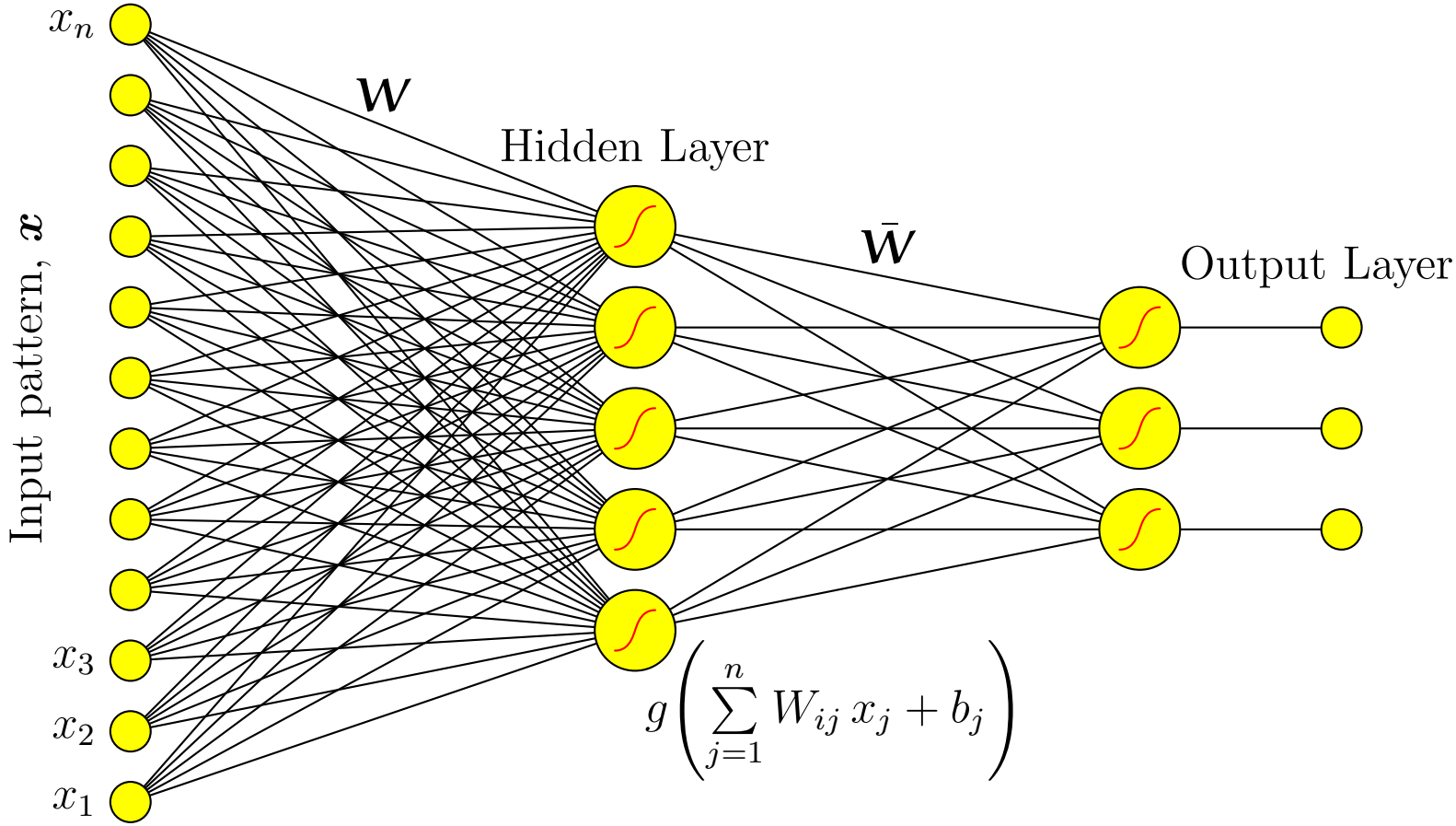
Multilayer Perceptron (MLP)



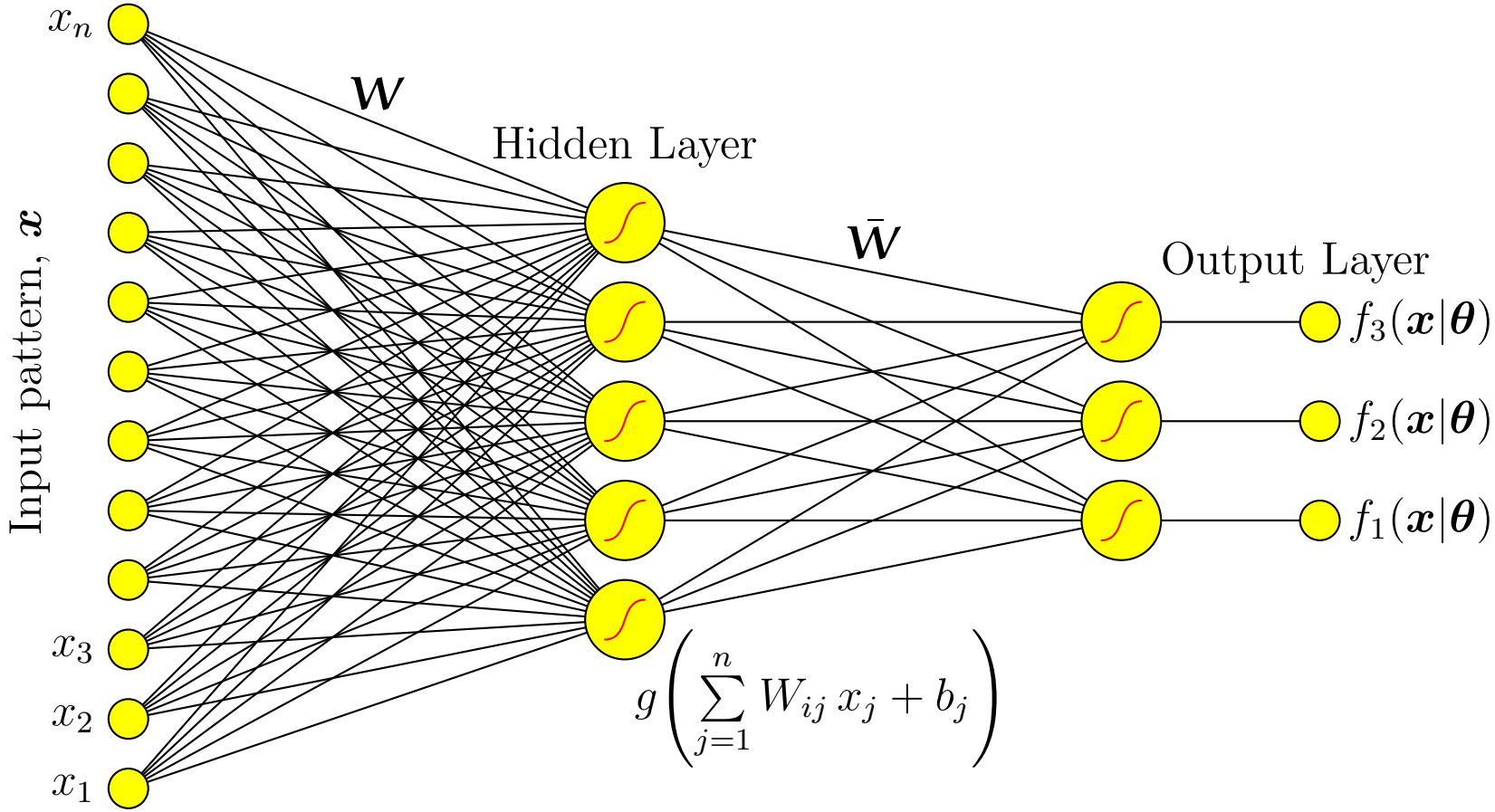
Multilayer Perceptron (MLP)



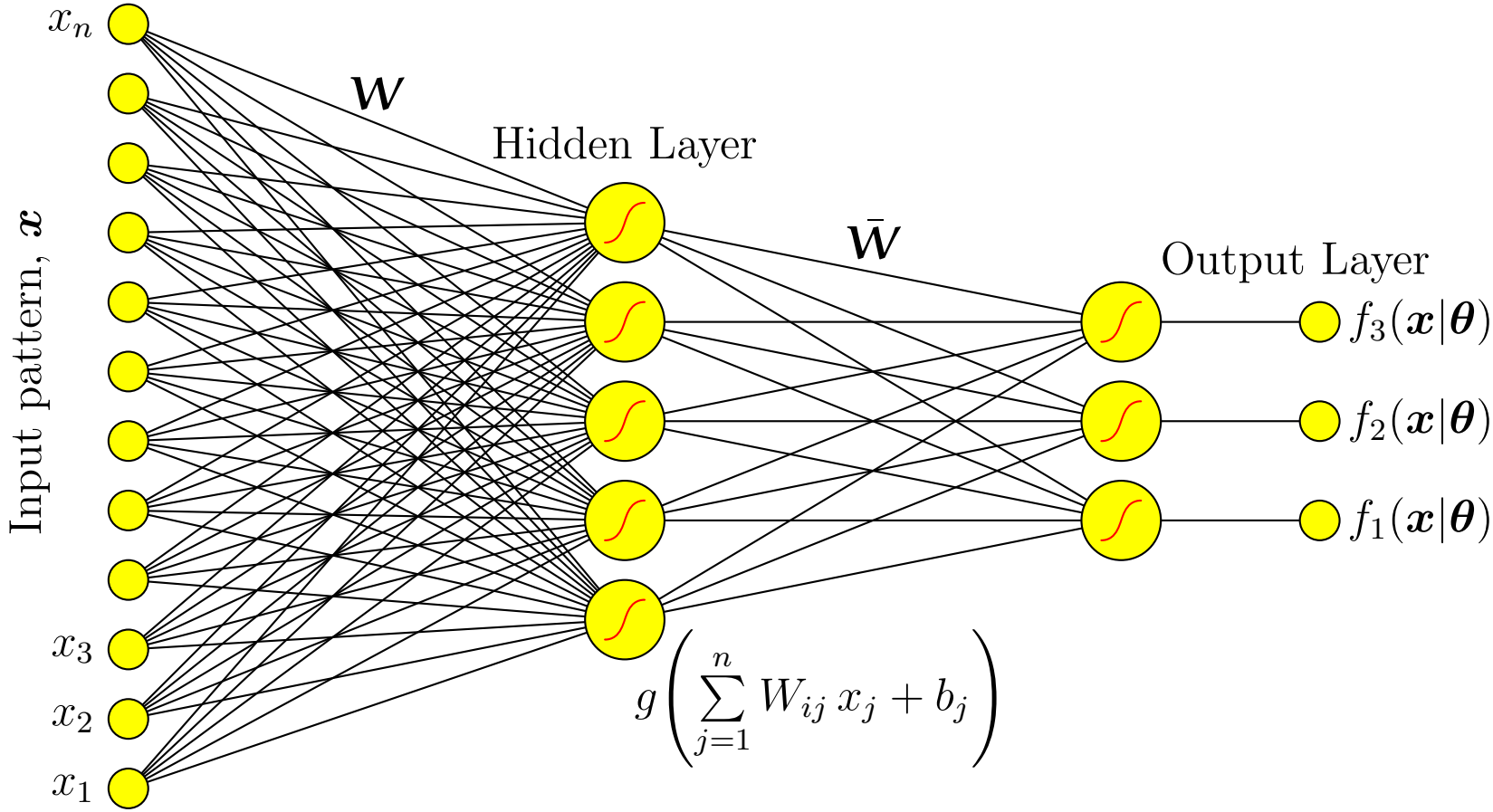
Multilayer Perceptron (MLP)



Multilayer Perceptron (MLP)

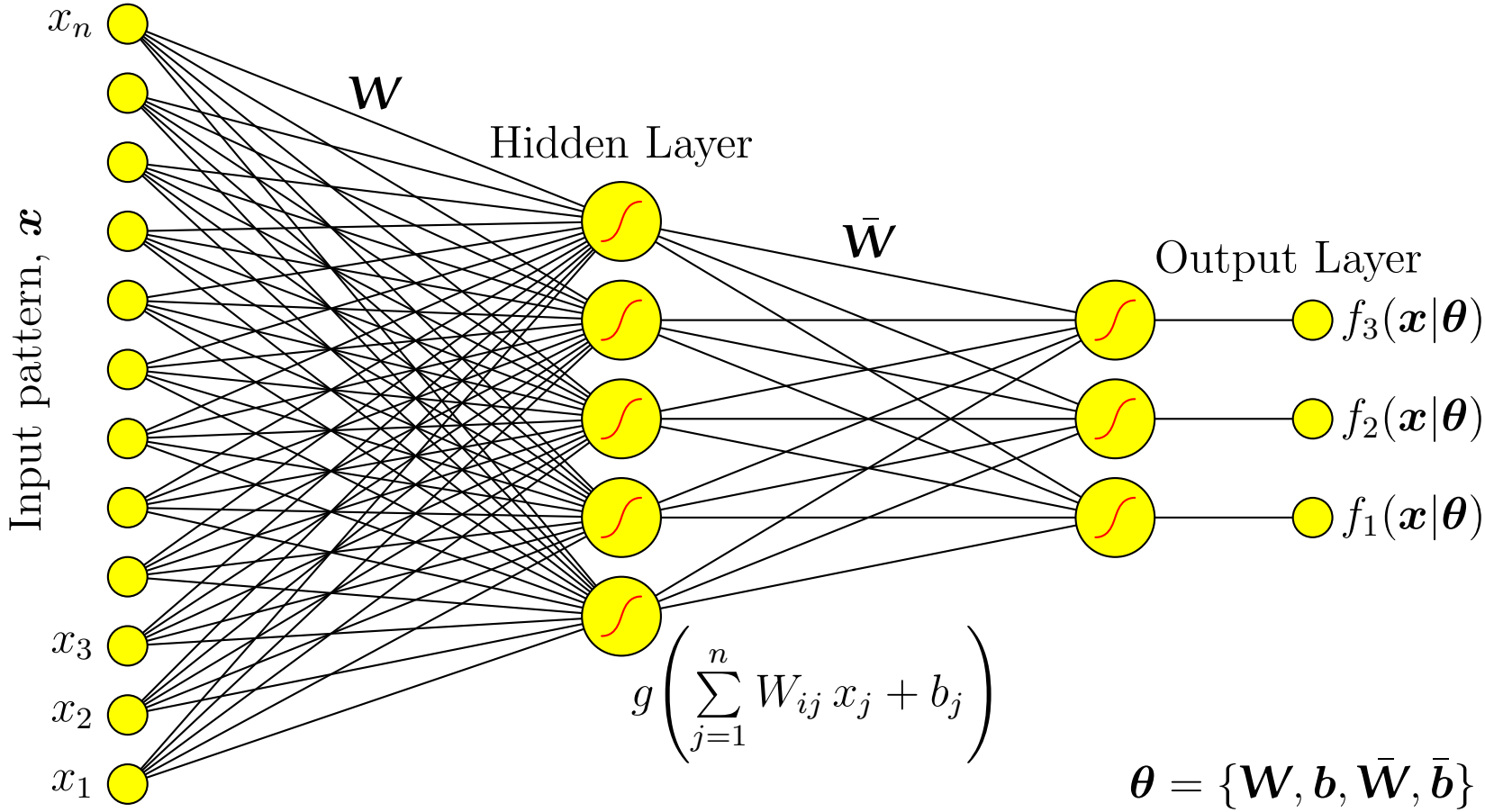


Multilayer Perceptron (MLP)



$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = g\left(\sum_{j=1}^{n_h} \bar{W}_{i,j}\, g\left(\sum_{k=1}^{n} W_{ik}\, x_k + b_k\right) + \bar{b}_i\right)$$

Multilayer Perceptron (MLP)



$$\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}, \bar{\boldsymbol{W}}, \bar{\boldsymbol{b}}\}$$

$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = g\left(\sum_{j=1}^{n_h} \bar{W}_{i,j}\, g\left(\sum_{k=1}^{n} W_{ik}\, x_k + b_k\right) + \bar{b}_i\right)$$

Training Neural Networks

- We train a neural network on a set of data

$$\mathcal{D} = \left((\boldsymbol{x}^\mu, \boldsymbol{y}^\mu) \,\middle|\, \mu = 1,\, 2,\, \ldots,\, m \right)$$

- Define an loss function $L(\boldsymbol{\theta}, \mathcal{D})$

- E.g. squared error

$$L(\boldsymbol{\theta}, \mathcal{D}) = \frac{1}{m} \sum_{\mu=1}^{m} \left\| \boldsymbol{y}^\mu - \boldsymbol{f}(\boldsymbol{x}^\mu | \boldsymbol{\theta}) \right\|^2$$

Training Neural Networks

- We train a neural network on a set of data

$$\mathcal{D} = \left((\boldsymbol{x}^\mu, \boldsymbol{y}^\mu) \,\middle|\, \mu = 1,\, 2,\, \ldots,\, m \right)$$

- Define an loss function $L(\boldsymbol{\theta}, \mathcal{D})$

- E.g. squared error

$$L(\boldsymbol{\theta}, \mathcal{D}) = \frac{1}{m} \sum_{\mu=1}^{m} \left\| \boldsymbol{y}^\mu - \boldsymbol{f}(\boldsymbol{x}^\mu | \boldsymbol{\theta}) \right\|^2$$

Training Neural Networks

- We train a neural network on a set of data

$$\mathcal{D} = \left((\boldsymbol{x}^\mu, \boldsymbol{y}^\mu) \,\middle|\, \mu = 1,\, 2,\, \ldots,\, m \right)$$

- Define an loss function $L(\boldsymbol{\theta}, \mathcal{D})$

- E.g. squared error

$$L(\boldsymbol{\theta}, \mathcal{D}) = \frac{1}{m} \sum_{\mu=1}^{m} \left\| \boldsymbol{y}^\mu - \boldsymbol{f}(\boldsymbol{x}^\mu | \boldsymbol{\theta}) \right\|^2$$

Gradient Descent

- To minimise the loss we follow the negative gradient

$$\boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D}) = \begin{pmatrix} \frac{\partial L(\boldsymbol{\theta}, \mathcal{D})}{\partial \theta_1} \\ \frac{\partial L(\boldsymbol{\theta}, \mathcal{D})}{\partial \theta_2} \\ \vdots \\ \frac{\partial L(\boldsymbol{\theta}, \mathcal{D})}{\partial \theta_N} \end{pmatrix}$$

- where $\theta_1 = W_{11}$, $\theta_2 = W_{12}$, etc.

- We make a step in the direction of the negative gradient

$$\boldsymbol{\theta}(t+1) = \boldsymbol{\theta}(t) - r\, \boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D})$$

- r is learning rate

Gradient Descent

- To minimise the loss we follow the negative gradient

$$\boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D}) = \begin{pmatrix} \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_1} \\ \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_2} \\ \vdots \\ \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_N} \end{pmatrix}$$

- where $\theta_1 = W_{11}$, $\theta_2 = W_{12}$, etc.

- We make a step in the direction of the negative gradient

$$\boldsymbol{\theta}(t+1) = \boldsymbol{\theta}(t) - r\,\boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D})$$

- r is learning rate

Gradient Descent

- To minimise the loss we follow the negative gradient

$$\boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D}) = \begin{pmatrix} \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_1} \\ \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_2} \\ \vdots \\ \frac{\partial L(\boldsymbol{\theta},\mathcal{D})}{\partial \theta_N} \end{pmatrix}$$

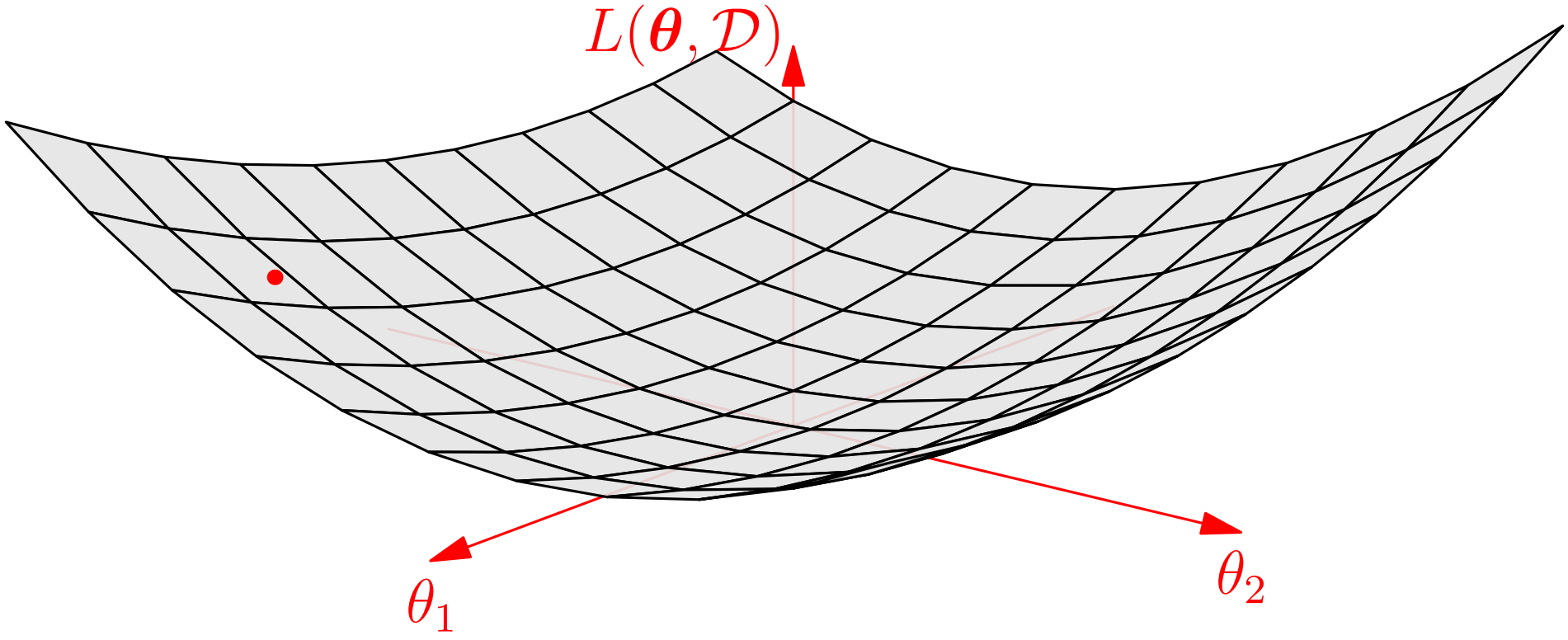
- where $\theta_1 = W_{11}$, $\theta_2 = W_{12}$, etc.

- We make a step in the direction of the negative gradient

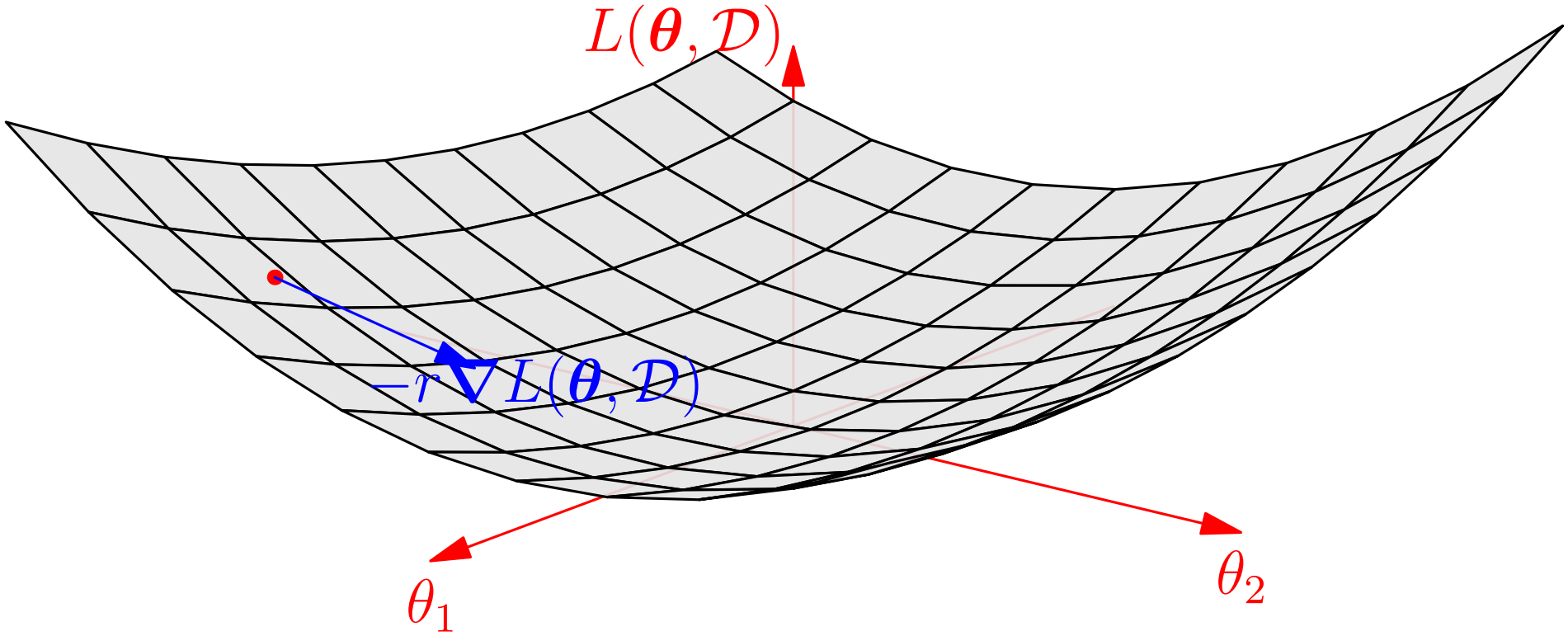
$$\boldsymbol{\theta}(t+1) = \boldsymbol{\theta}(t) - r\,\boldsymbol{\nabla} L(\boldsymbol{\theta}, \mathcal{D})$$

- r is learning rate

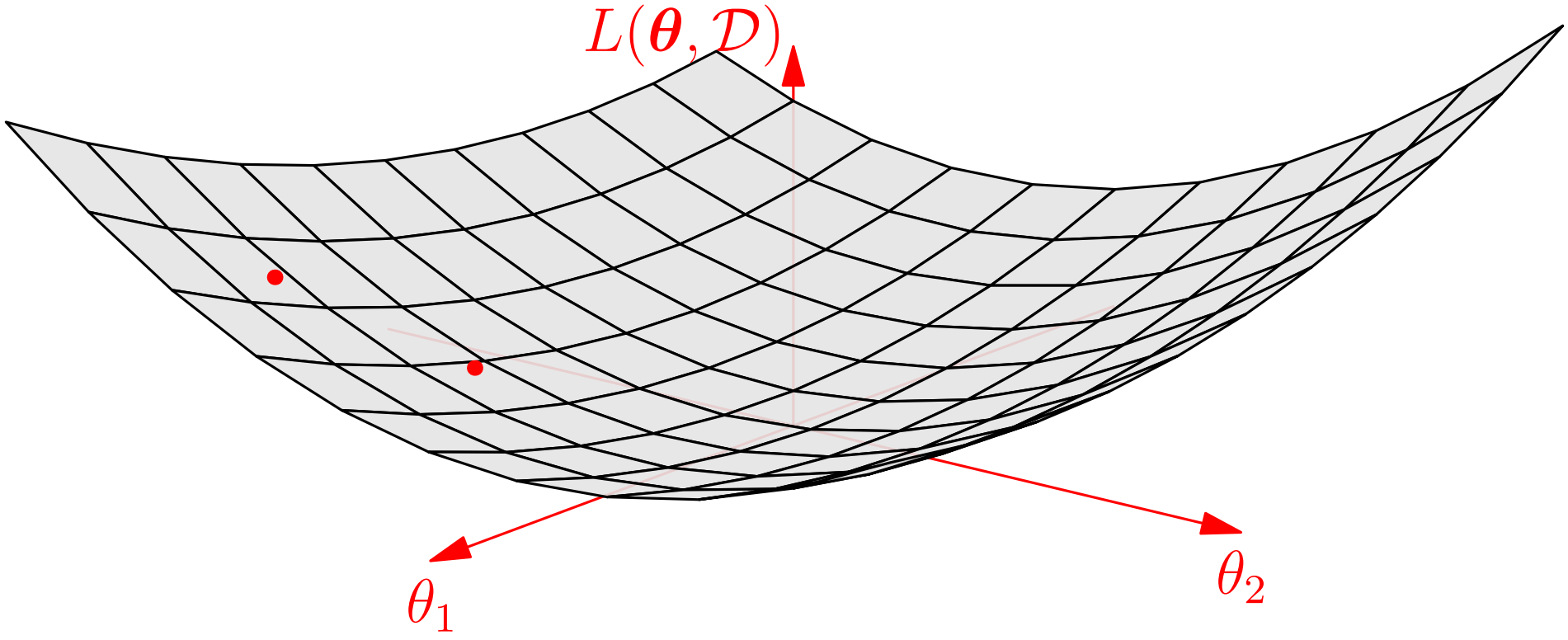
Following the Gradient



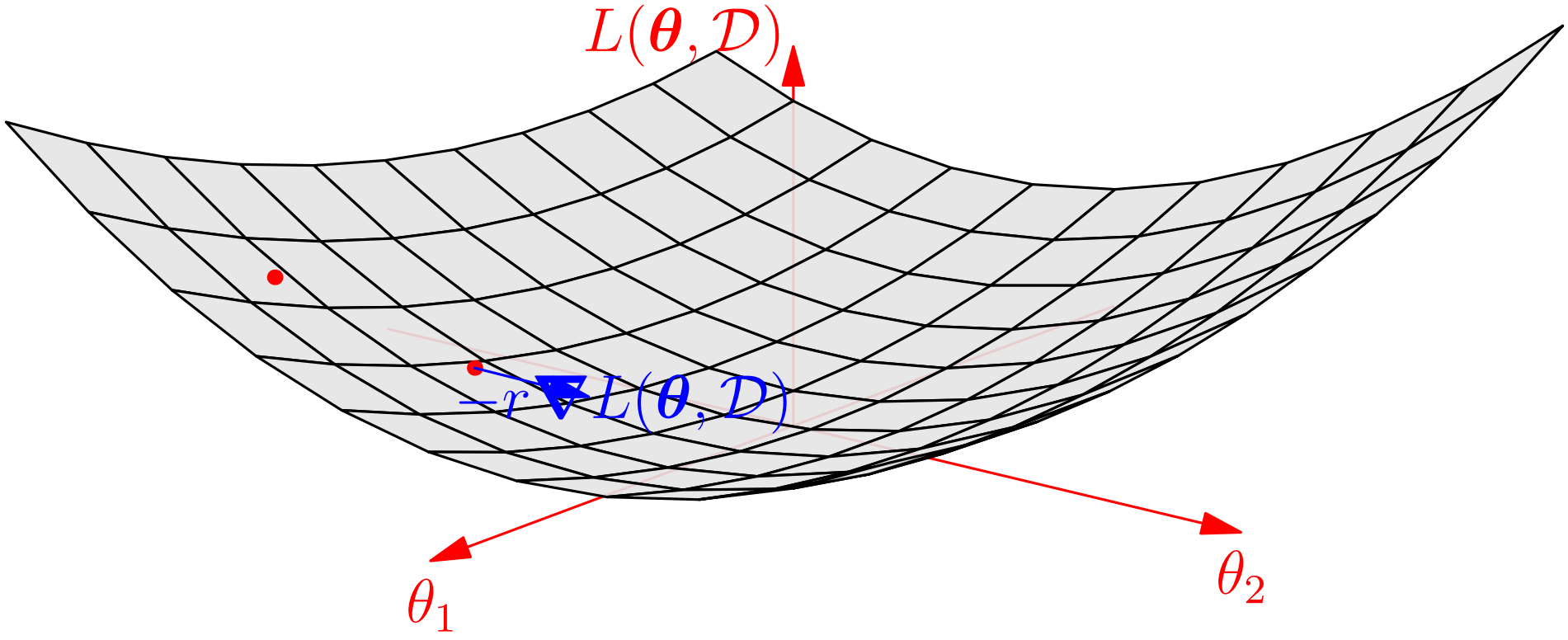
Following the Gradient



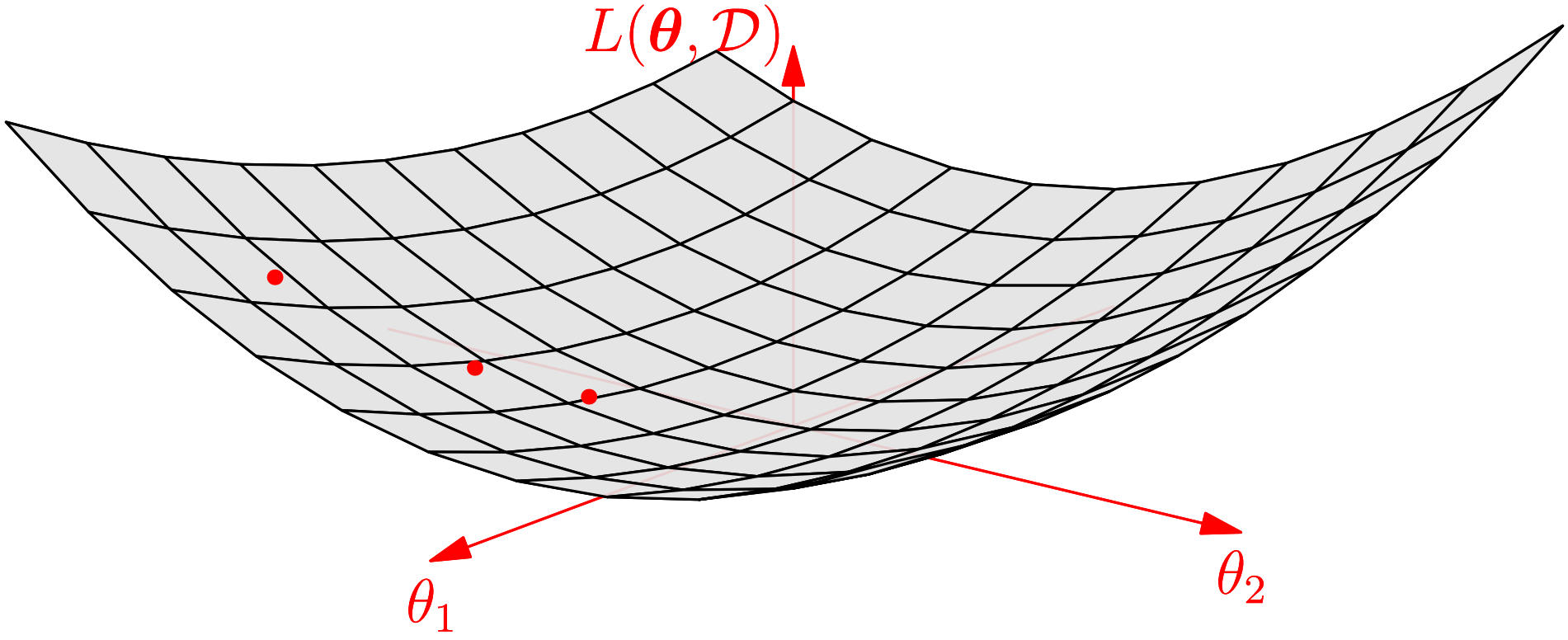
Following the Gradient



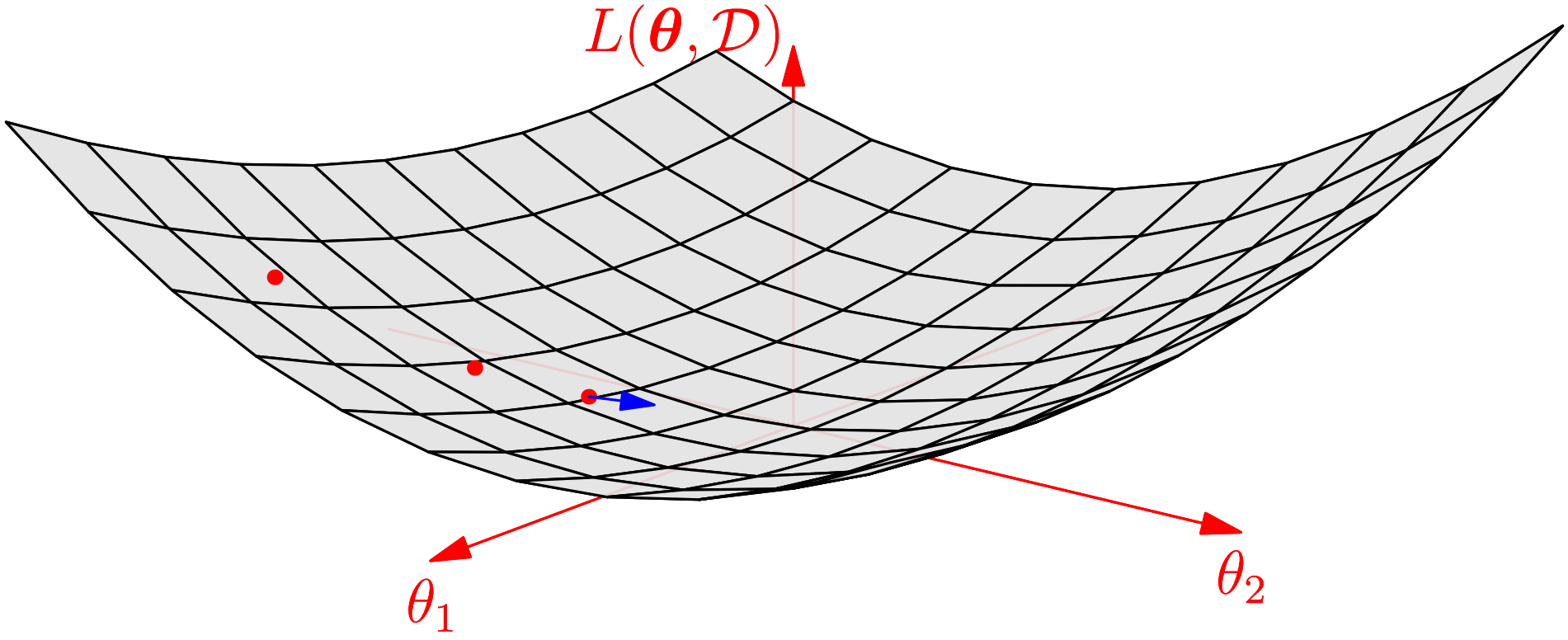
Following the Gradient



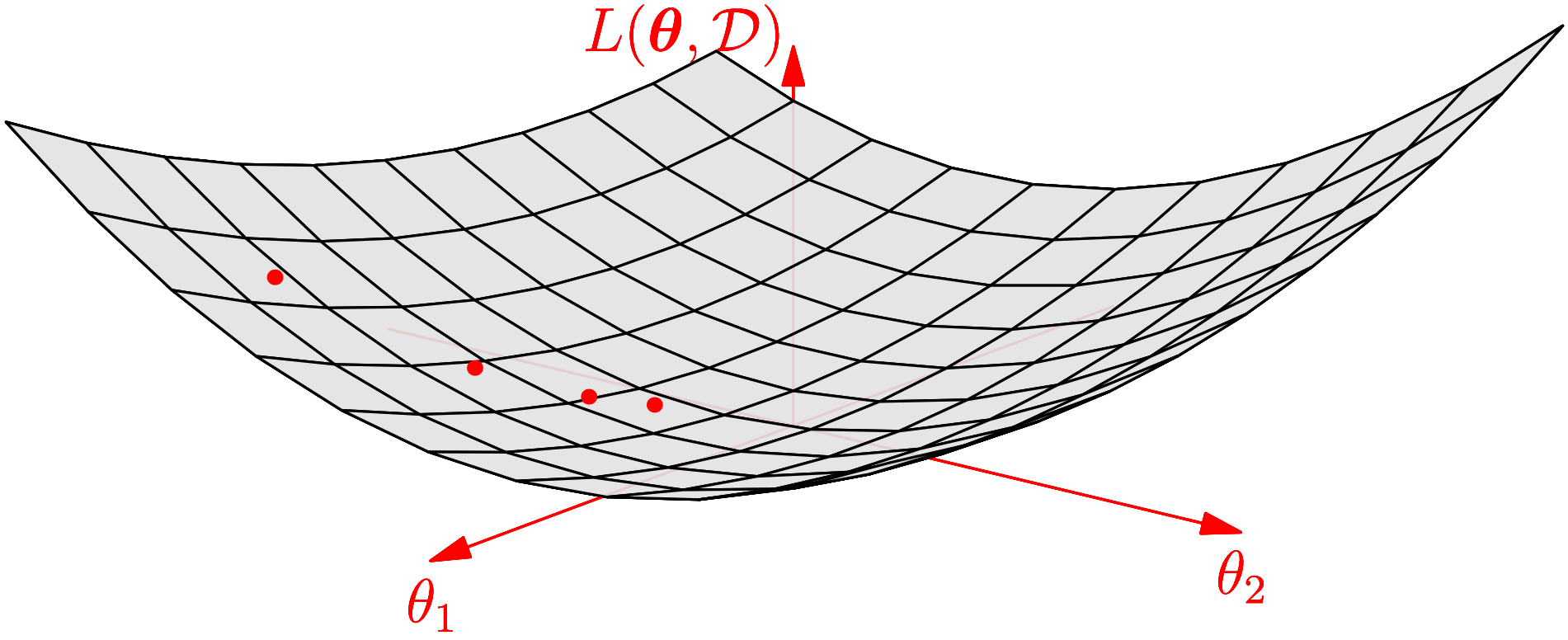
Following the Gradient



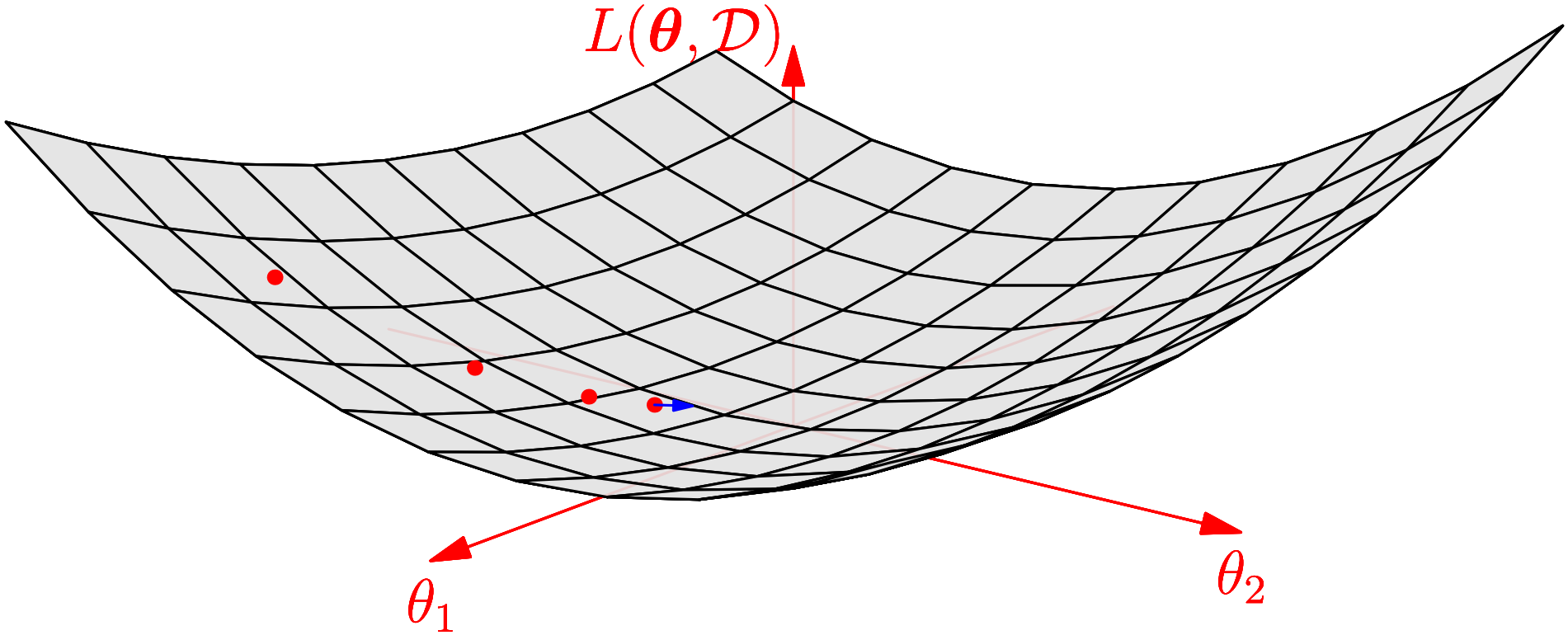
Following the Gradient



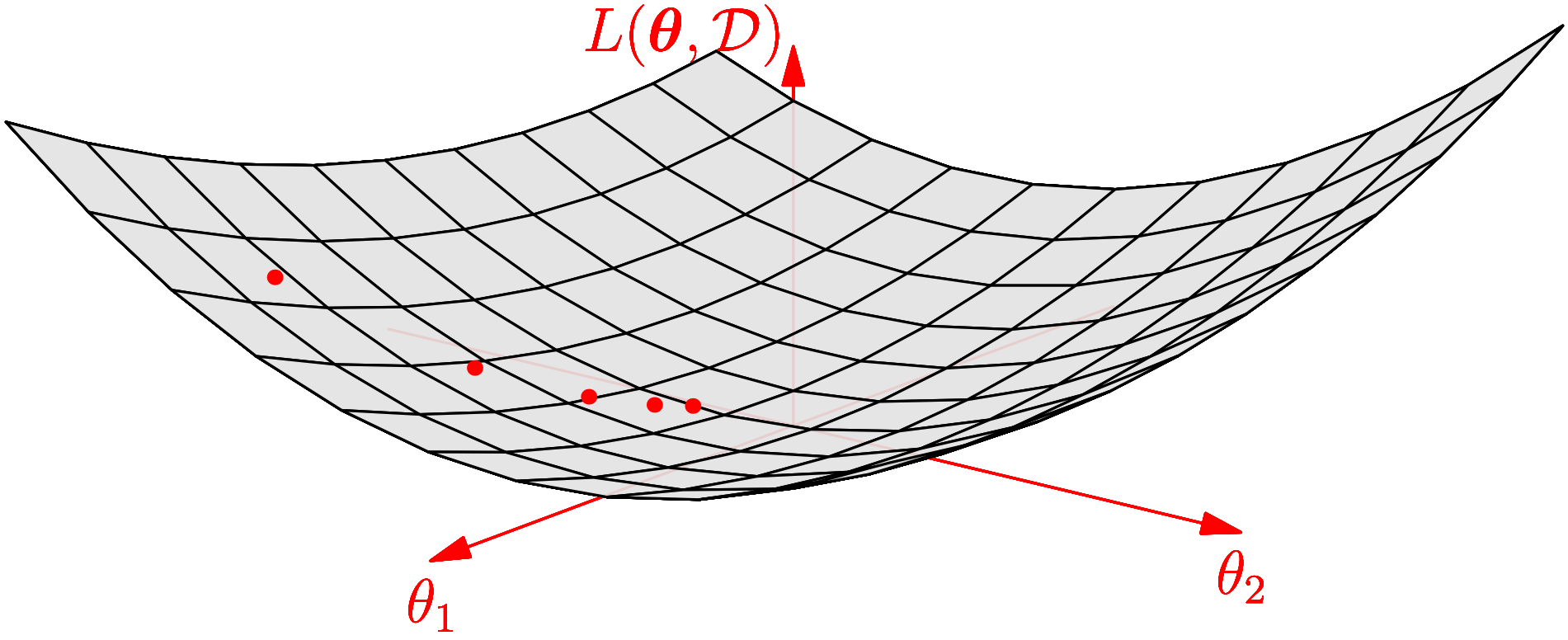
Following the Gradient



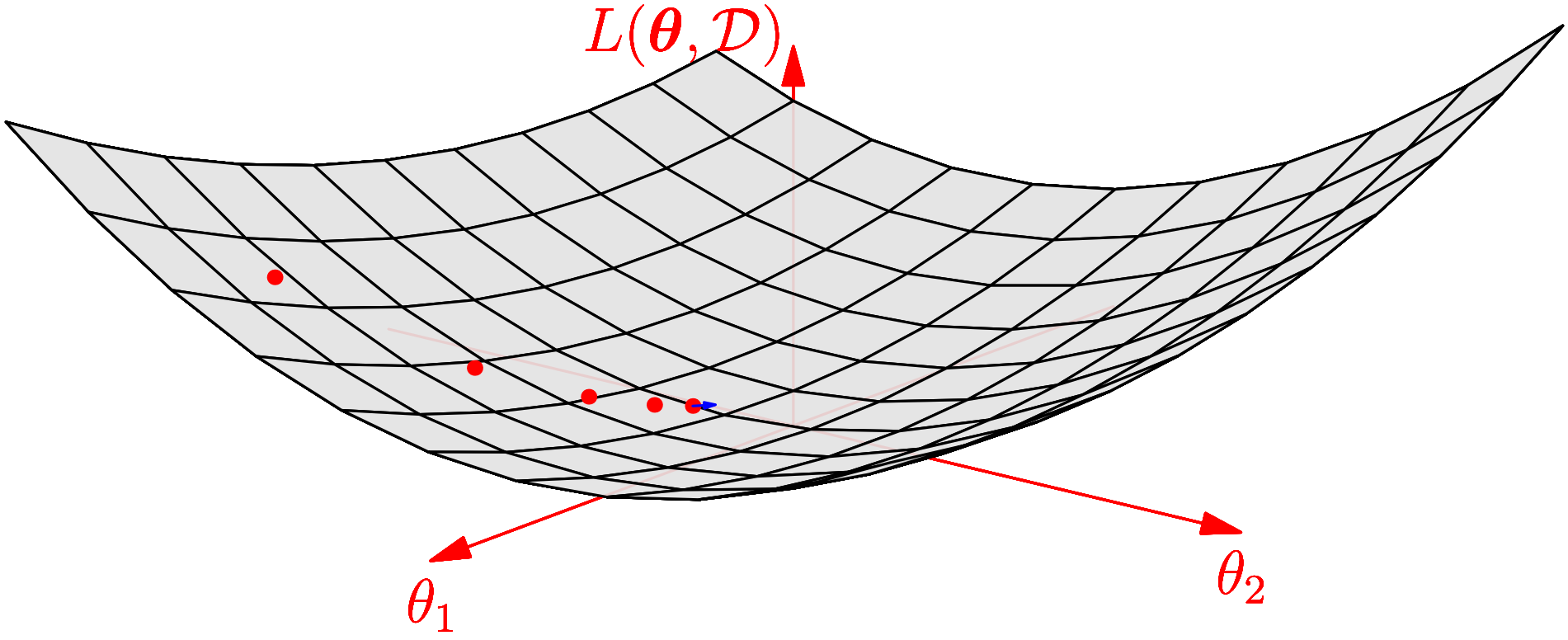
Following the Gradient



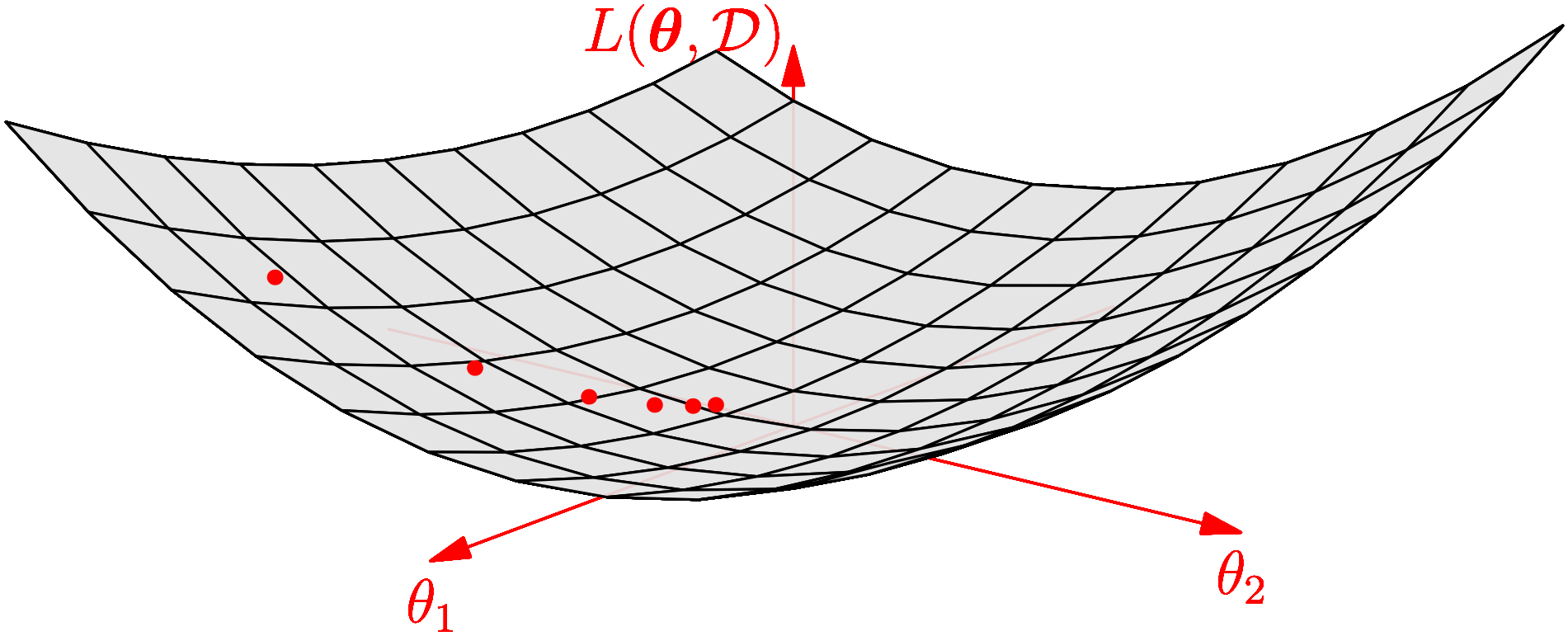
Following the Gradient



Following the Gradient



Following the Gradient



Backpropagation

- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

Backpropagation

- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

Backpropagation

- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

Backpropagation

- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

Backpropagation

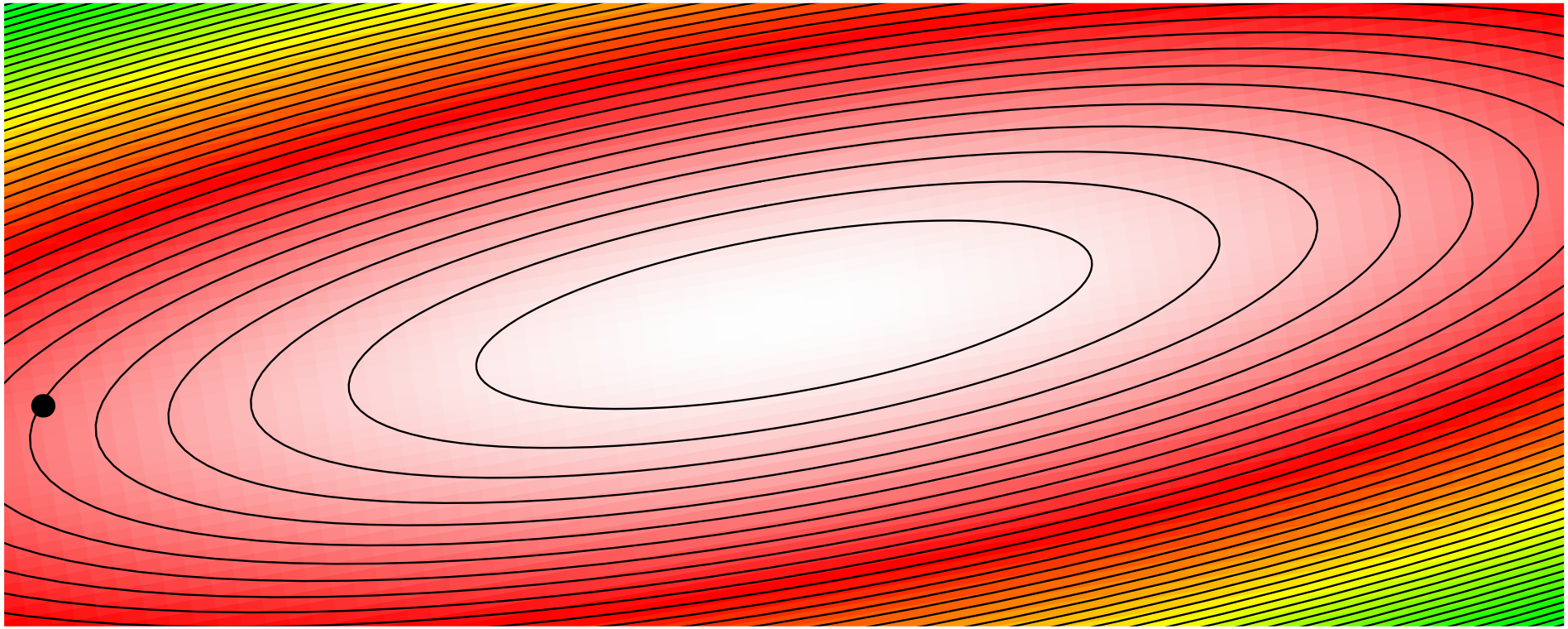
- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

Backpropagation

- Calculating the gradient for a MLP is known as back-propagation
- We can compute the gradient of the error for the whole training set
- In this case we often use Levenberg-Marquardt, scale conjugate gradient or BFGS to speed up convergence
- More often it is effective to find the error per input pattern or mini-batch (a small number of input patterns)
- This goes by the name of stochastic gradient descent
- This introduces noise which helps escape local optima

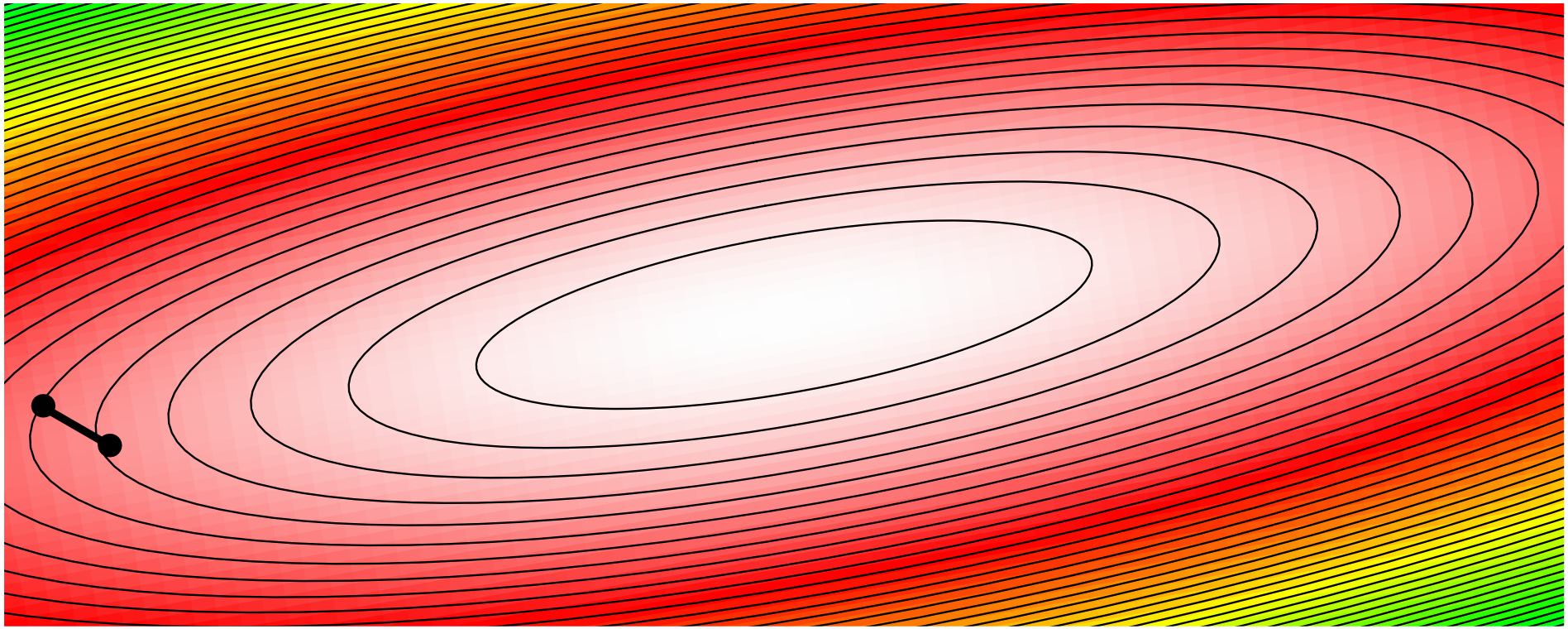
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



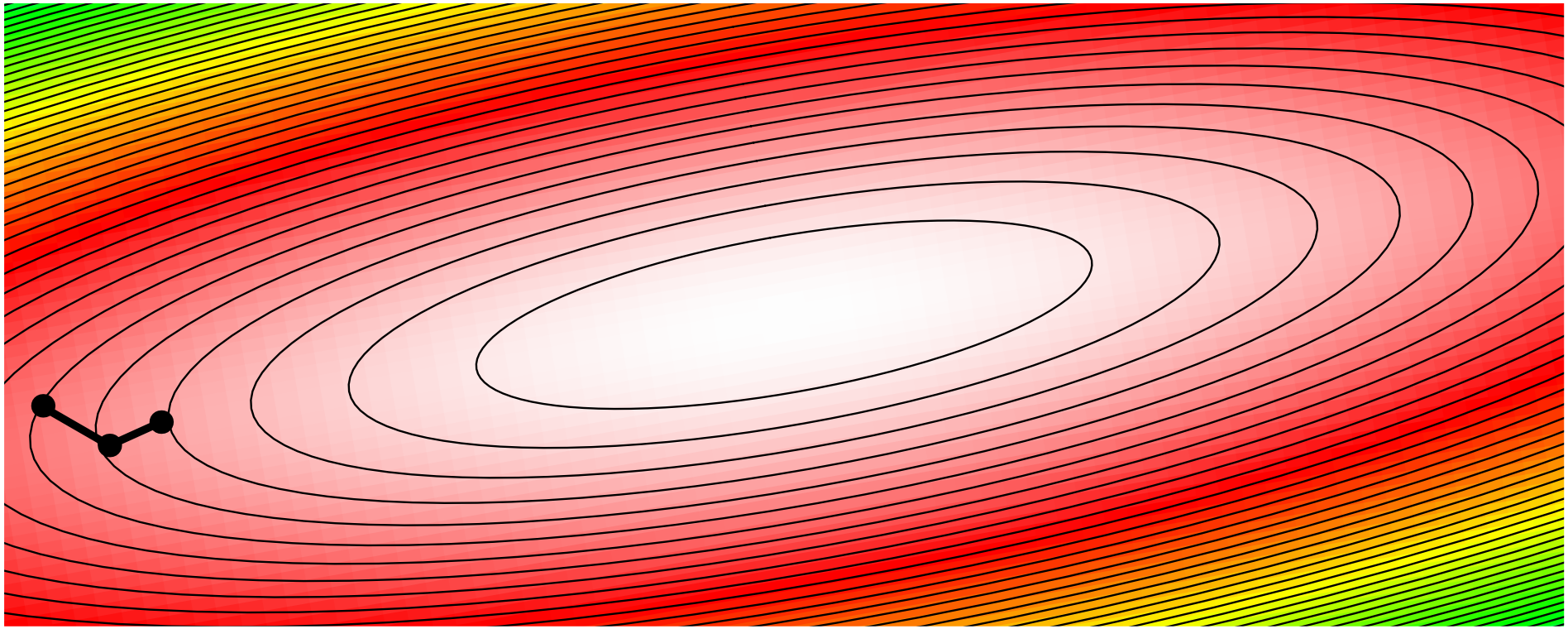
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



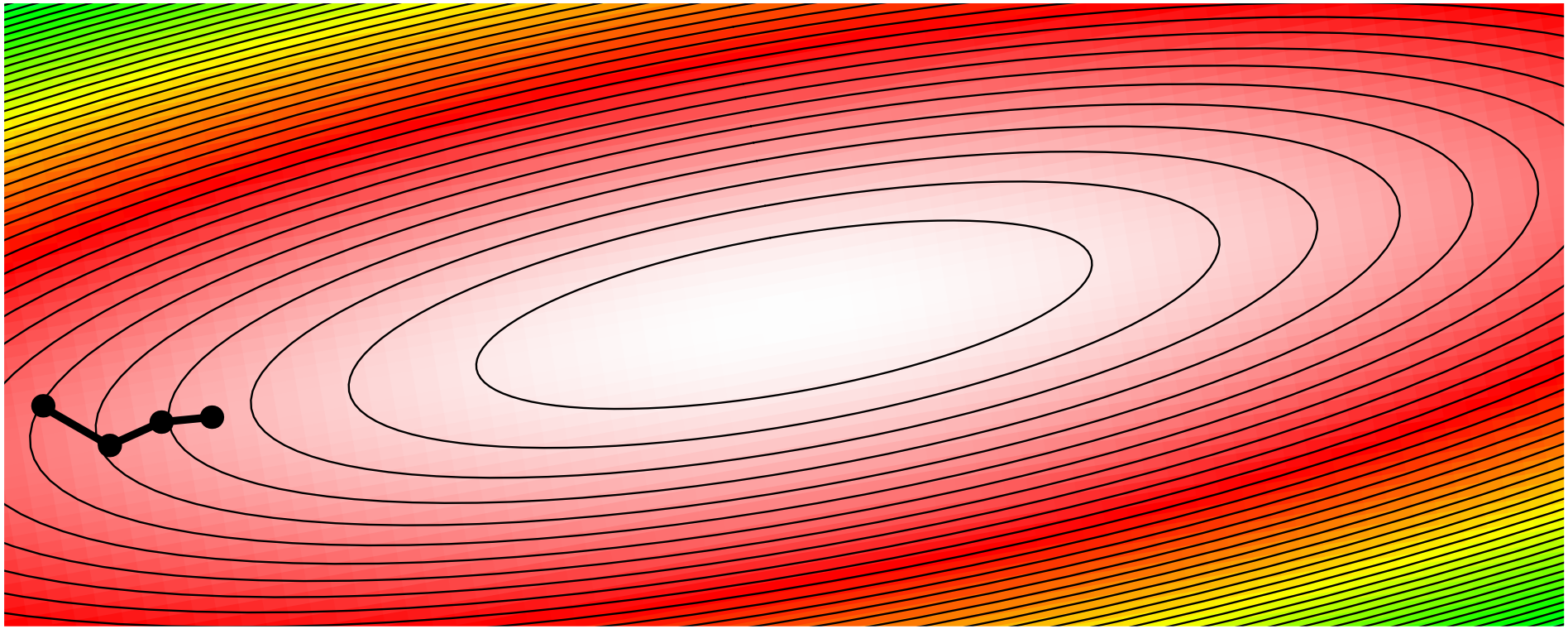
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



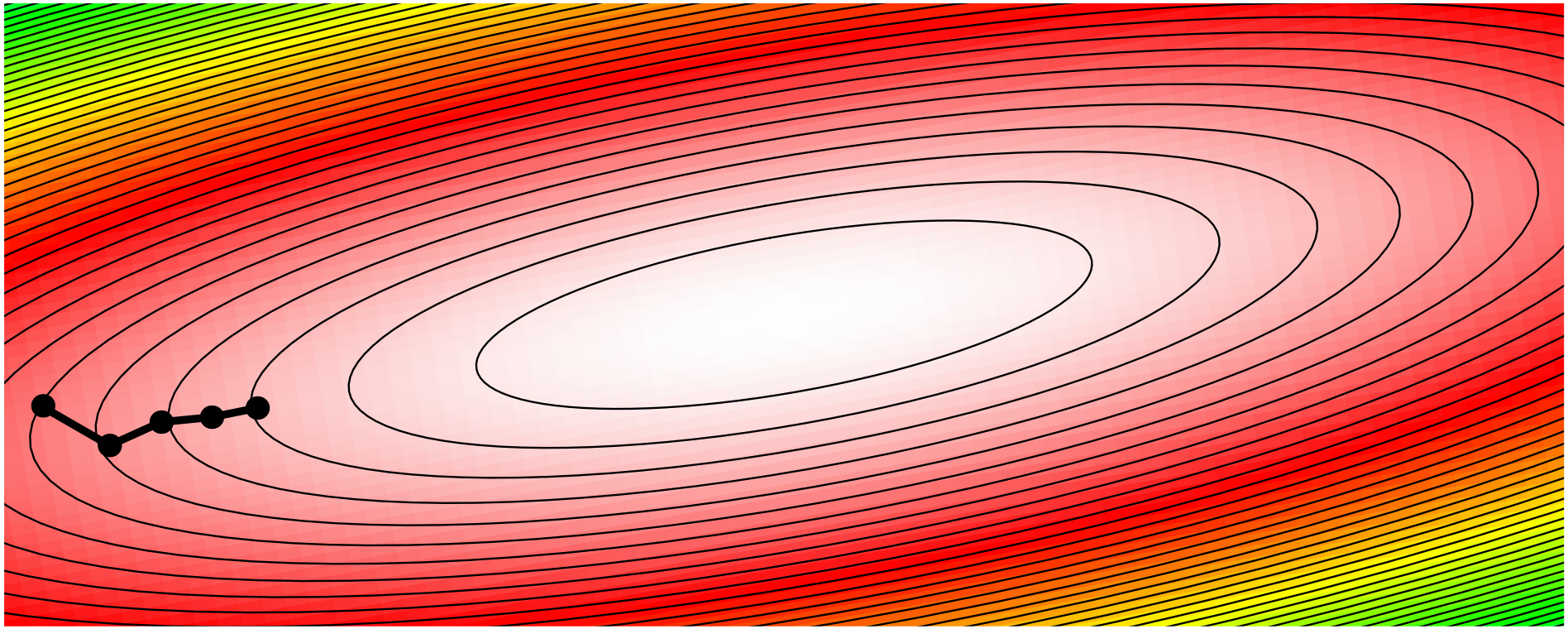
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



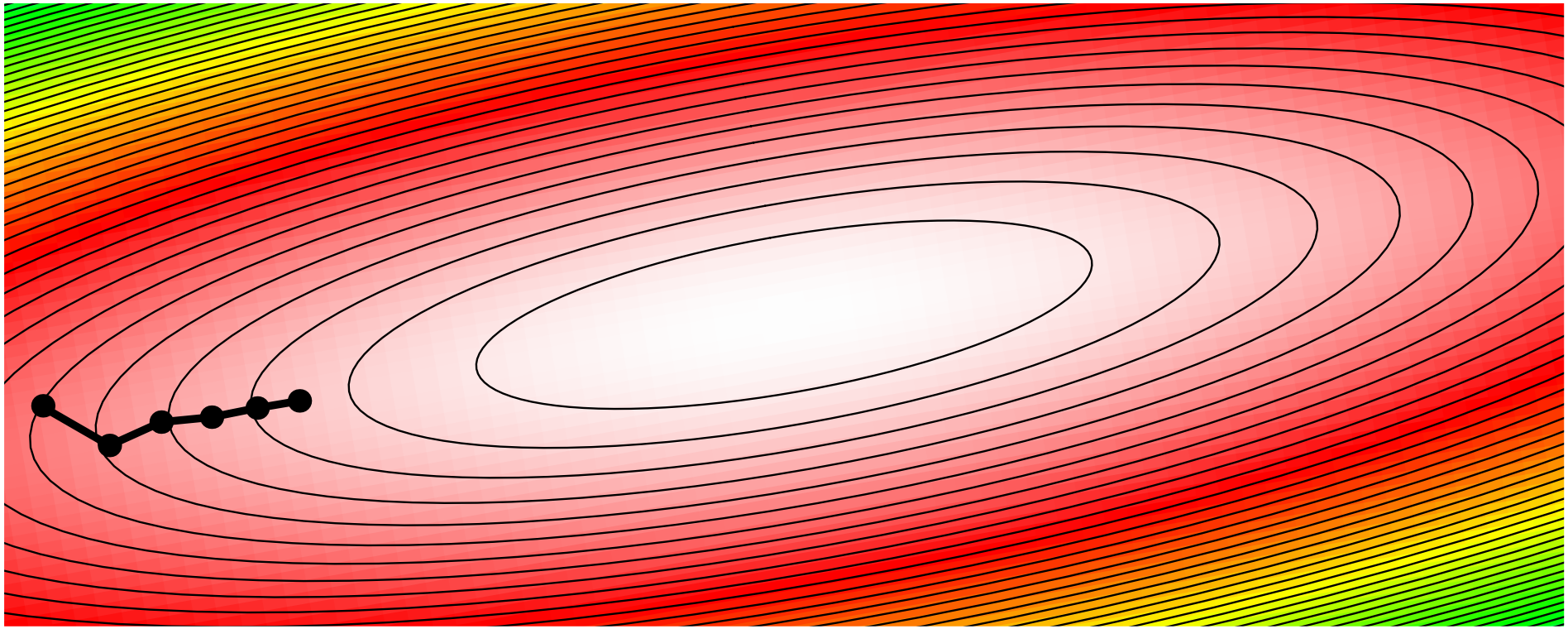
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



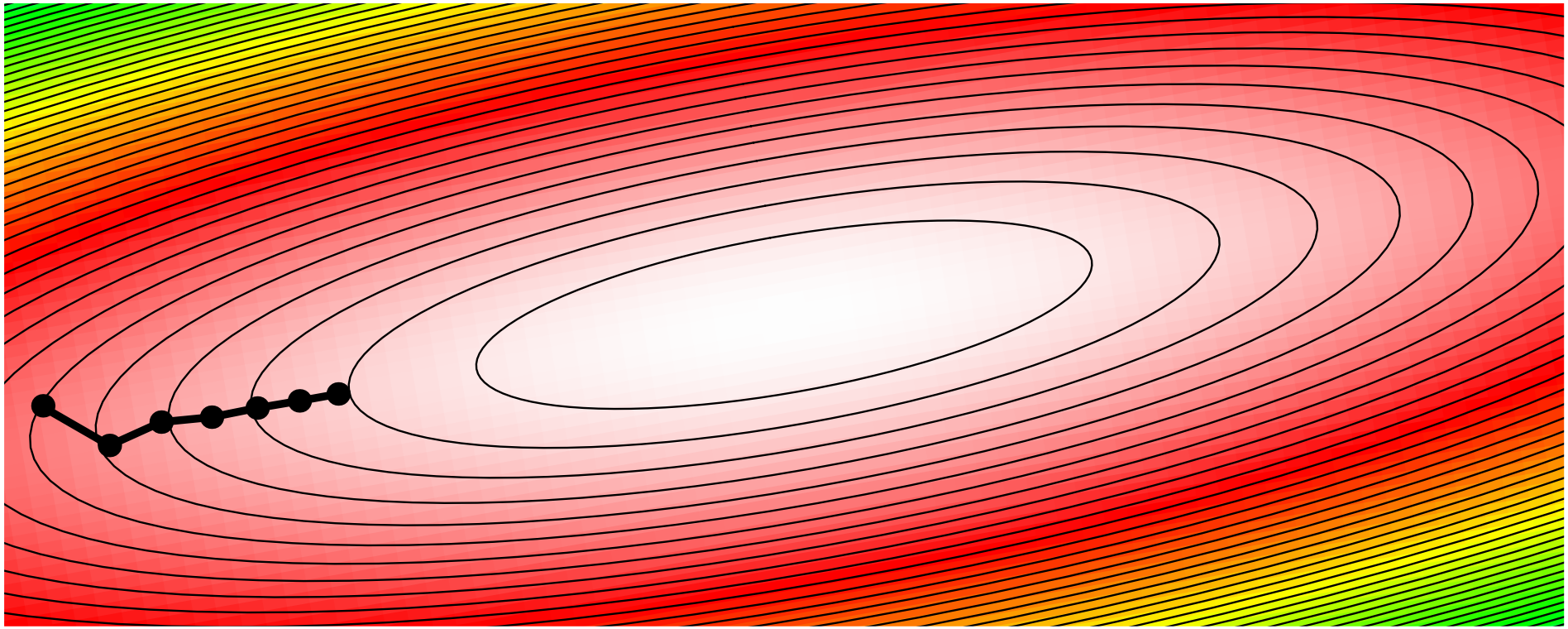
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



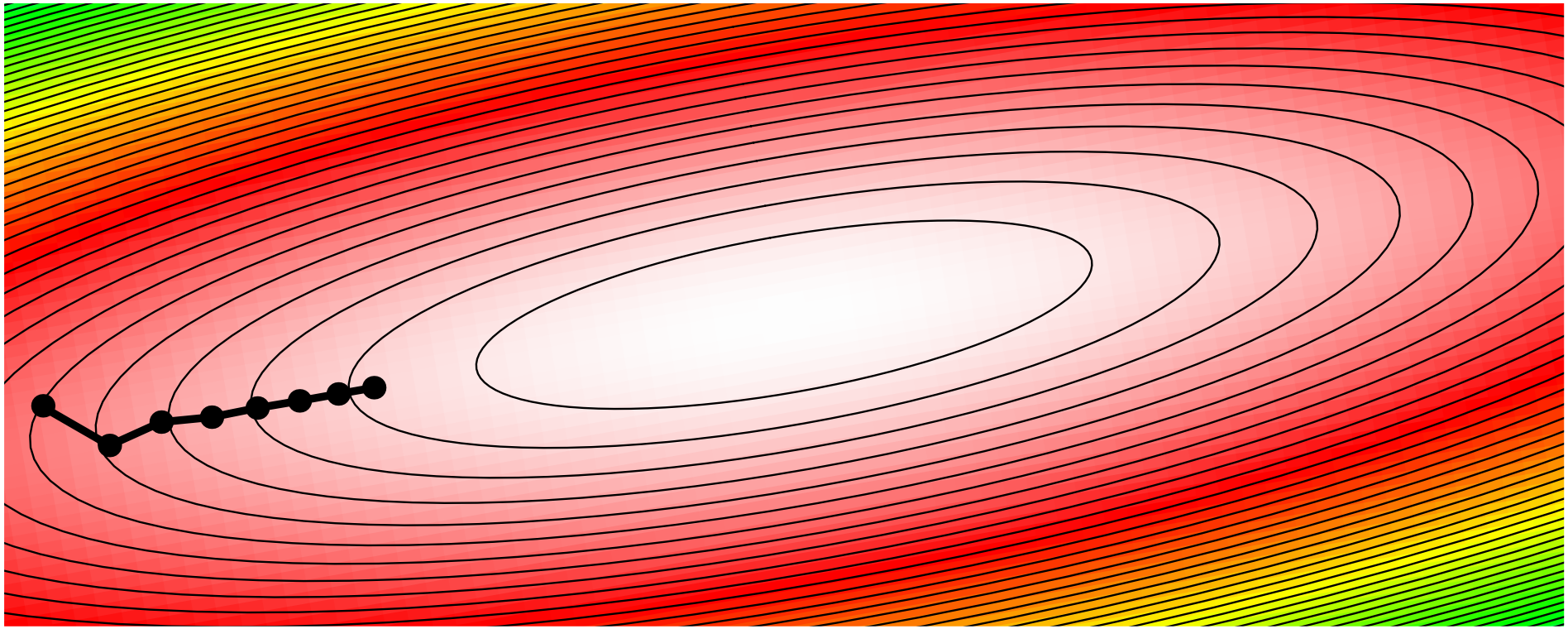
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



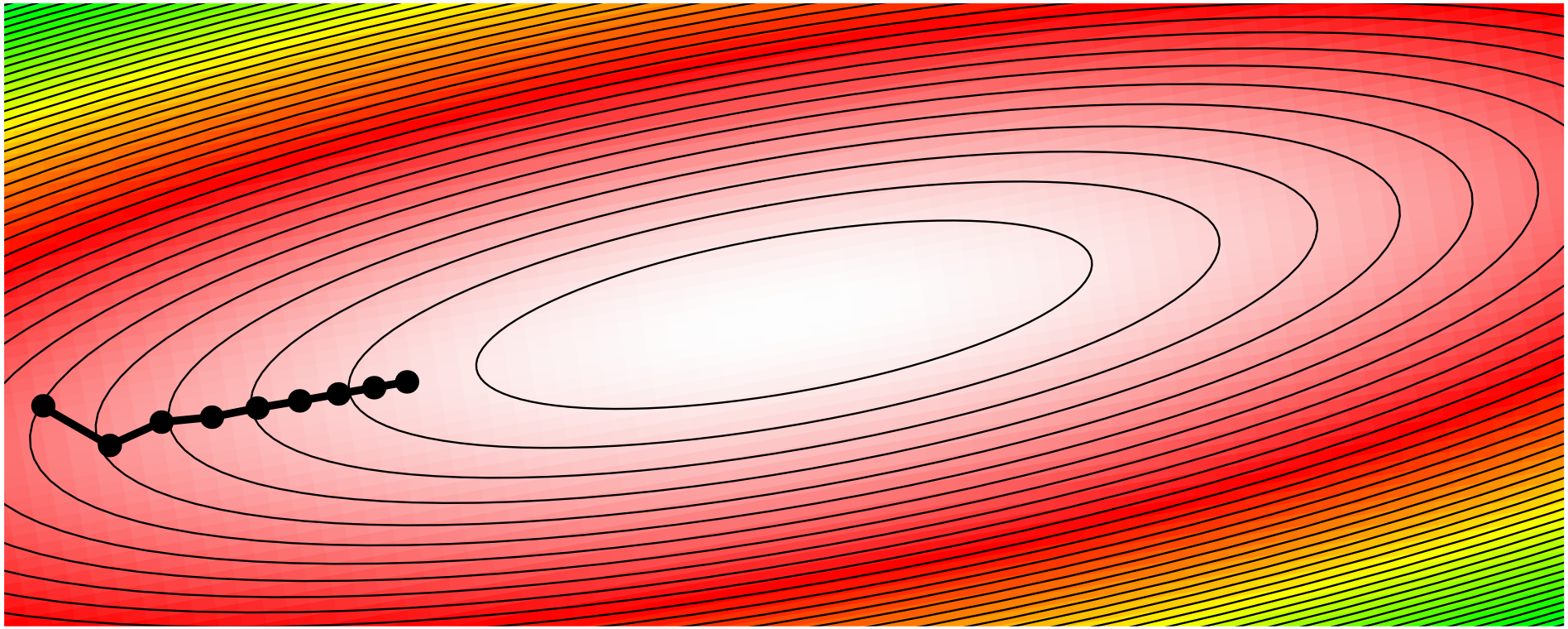
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



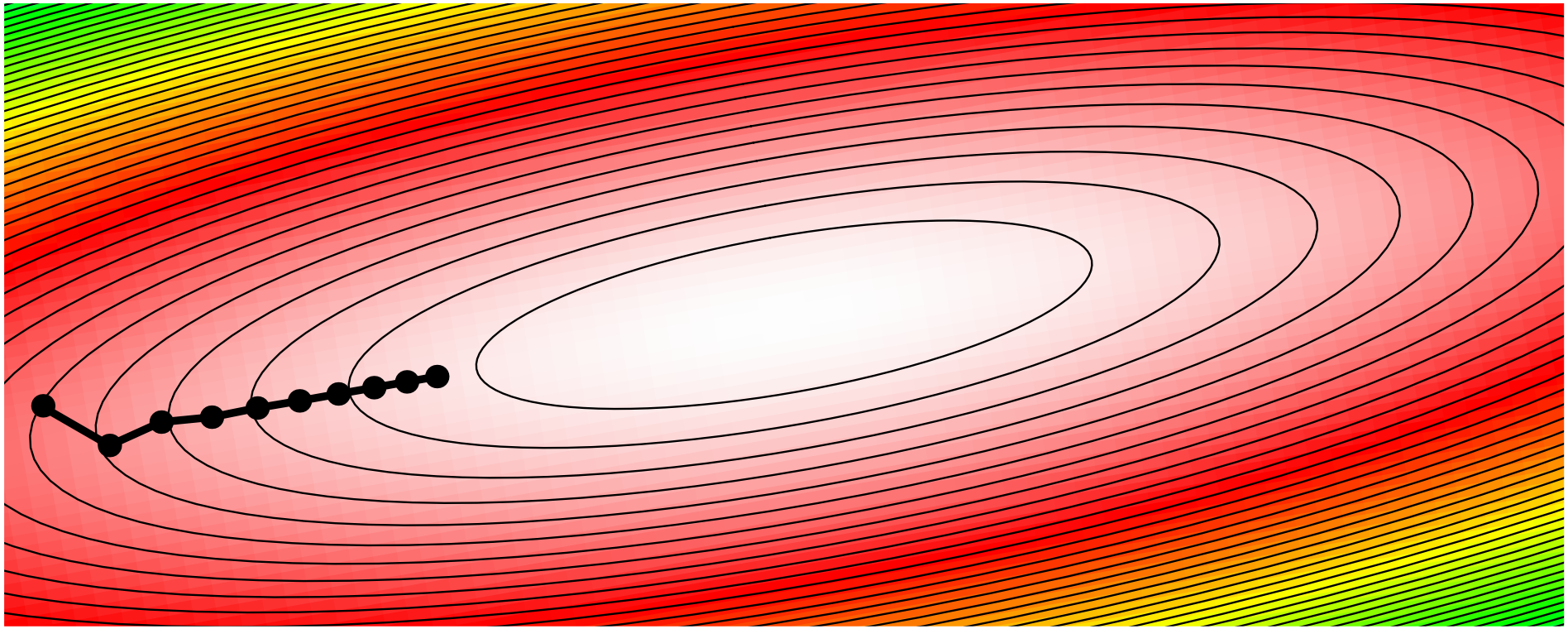
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



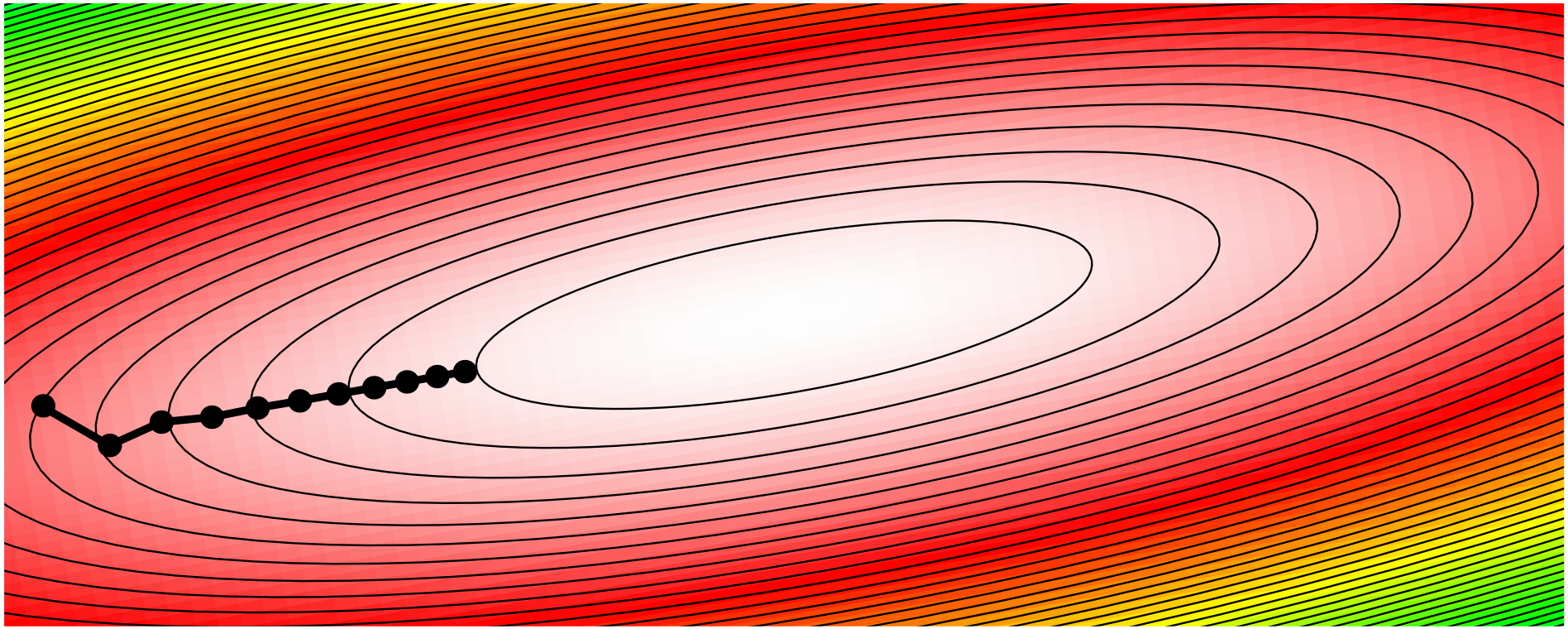
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way

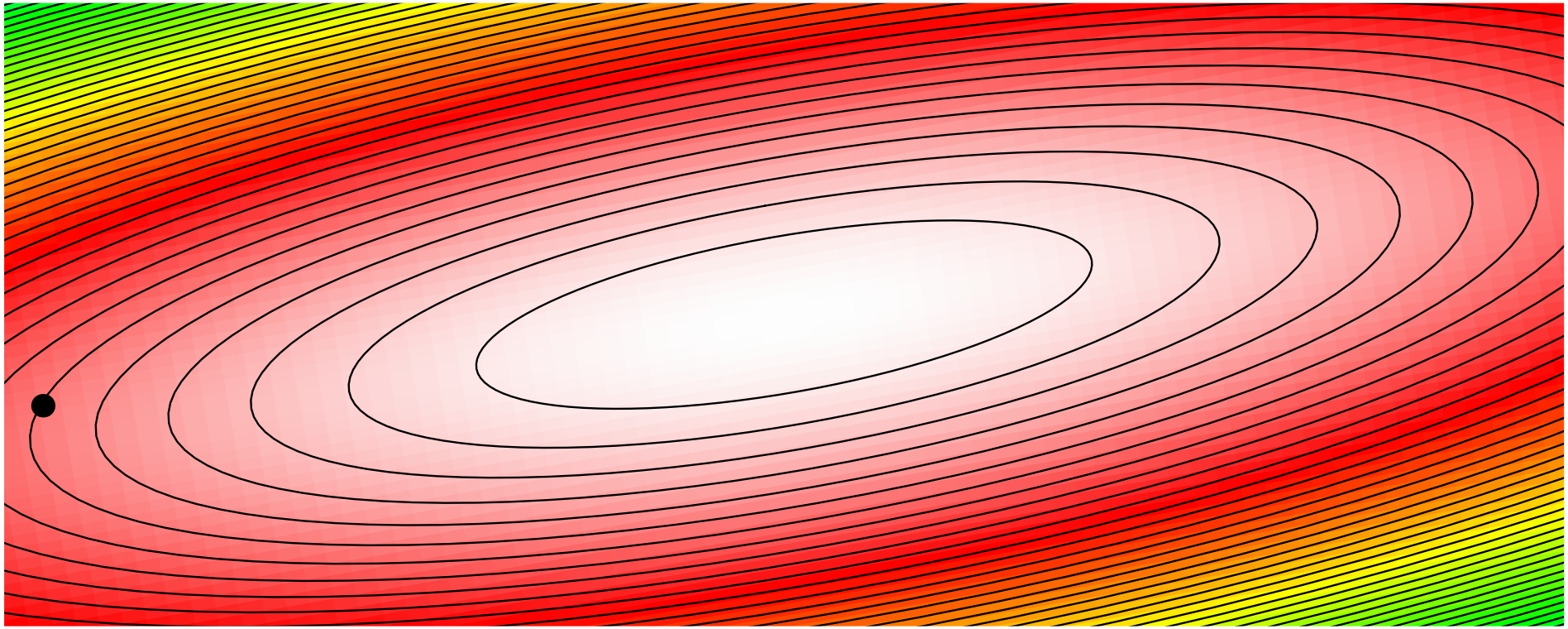


More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .

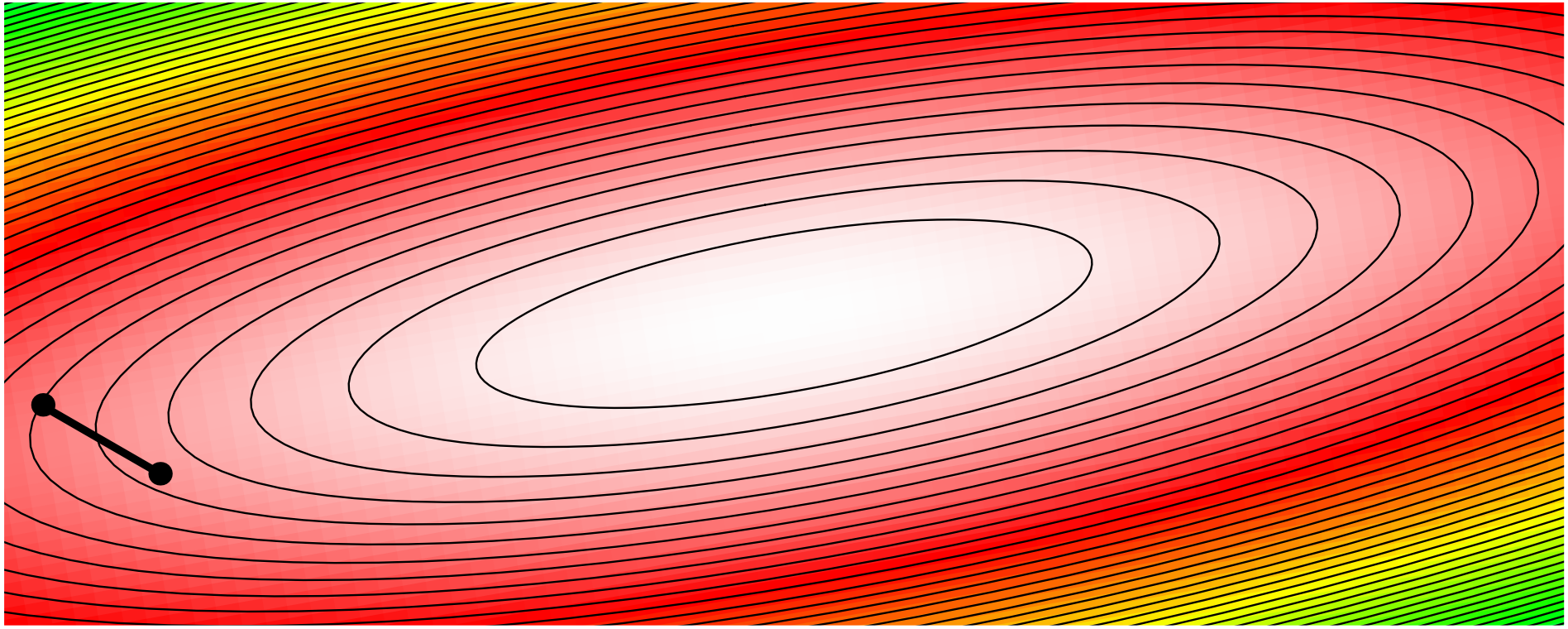
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



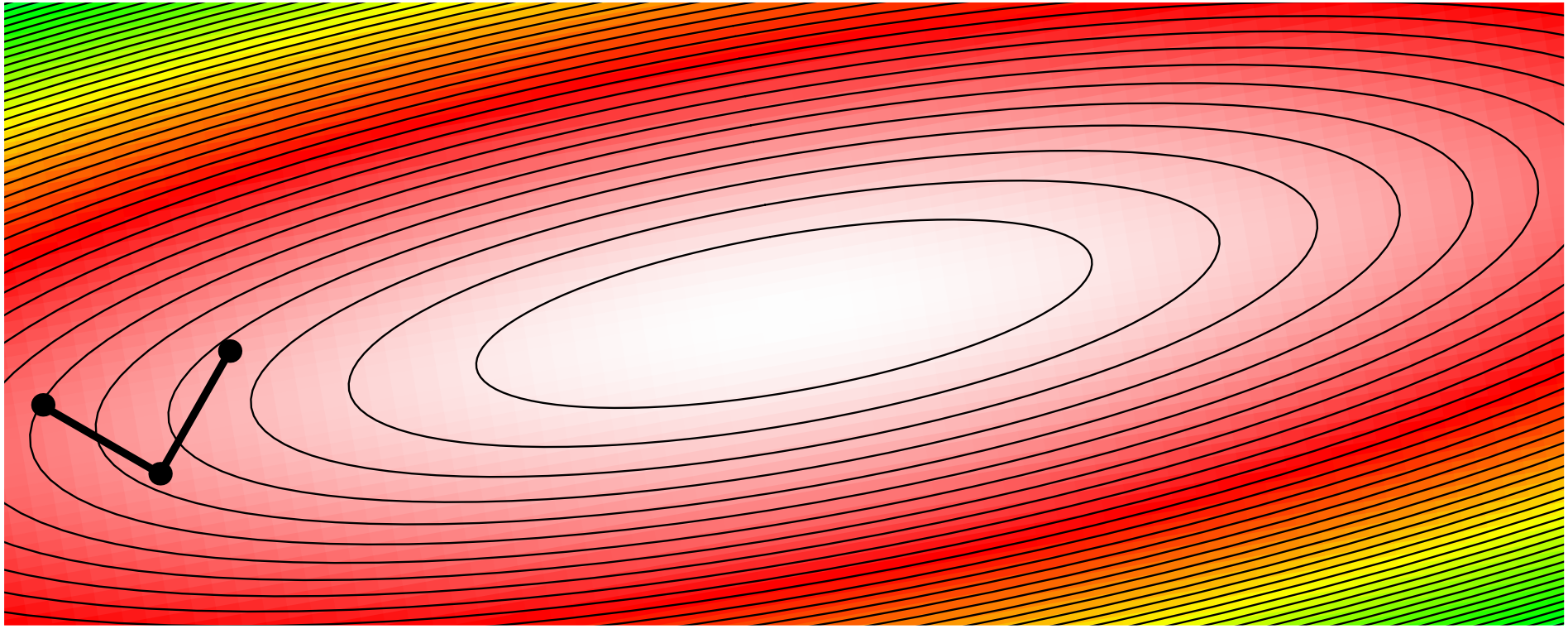
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



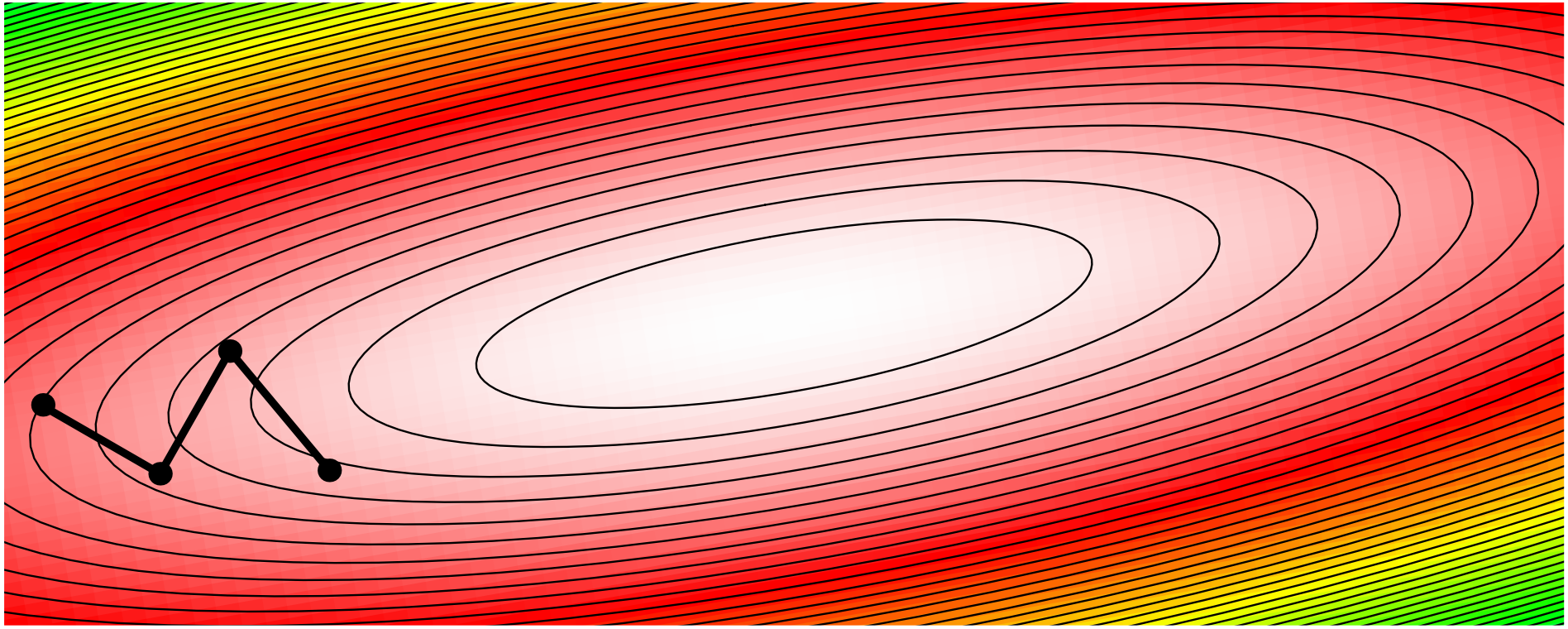
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



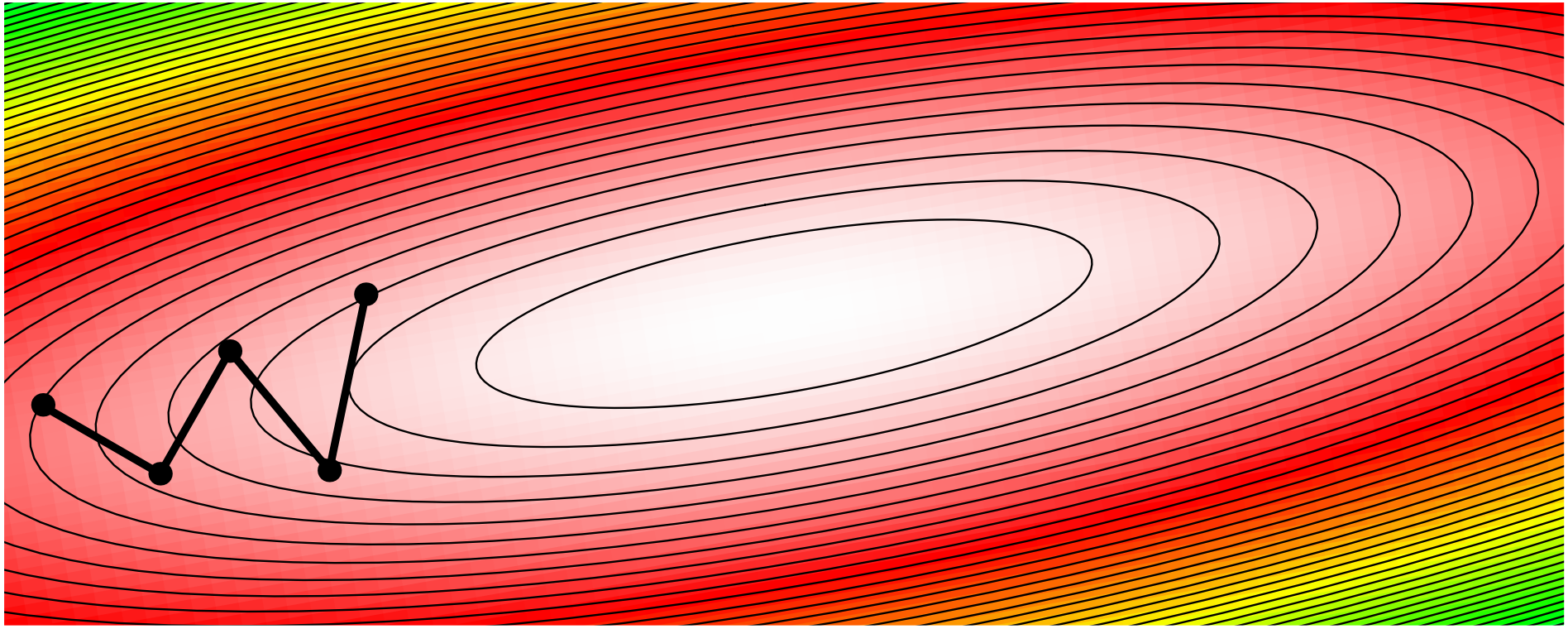
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



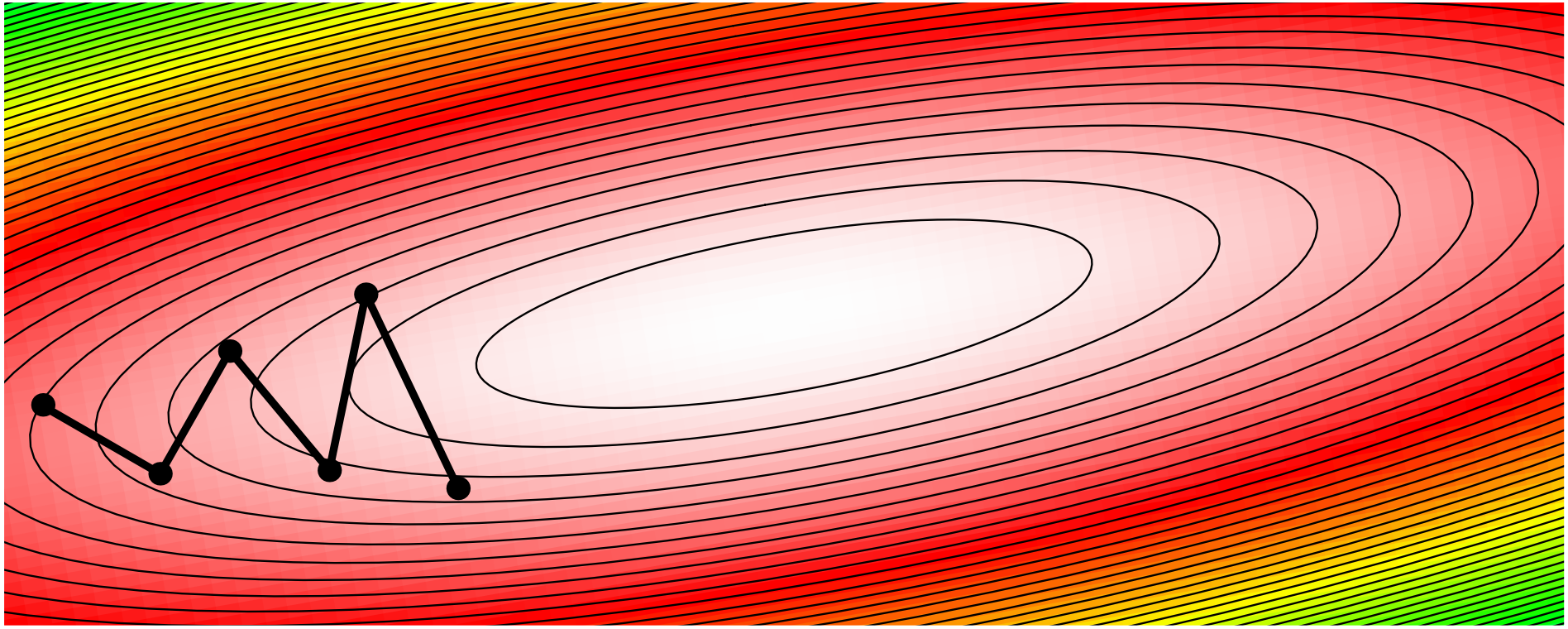
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



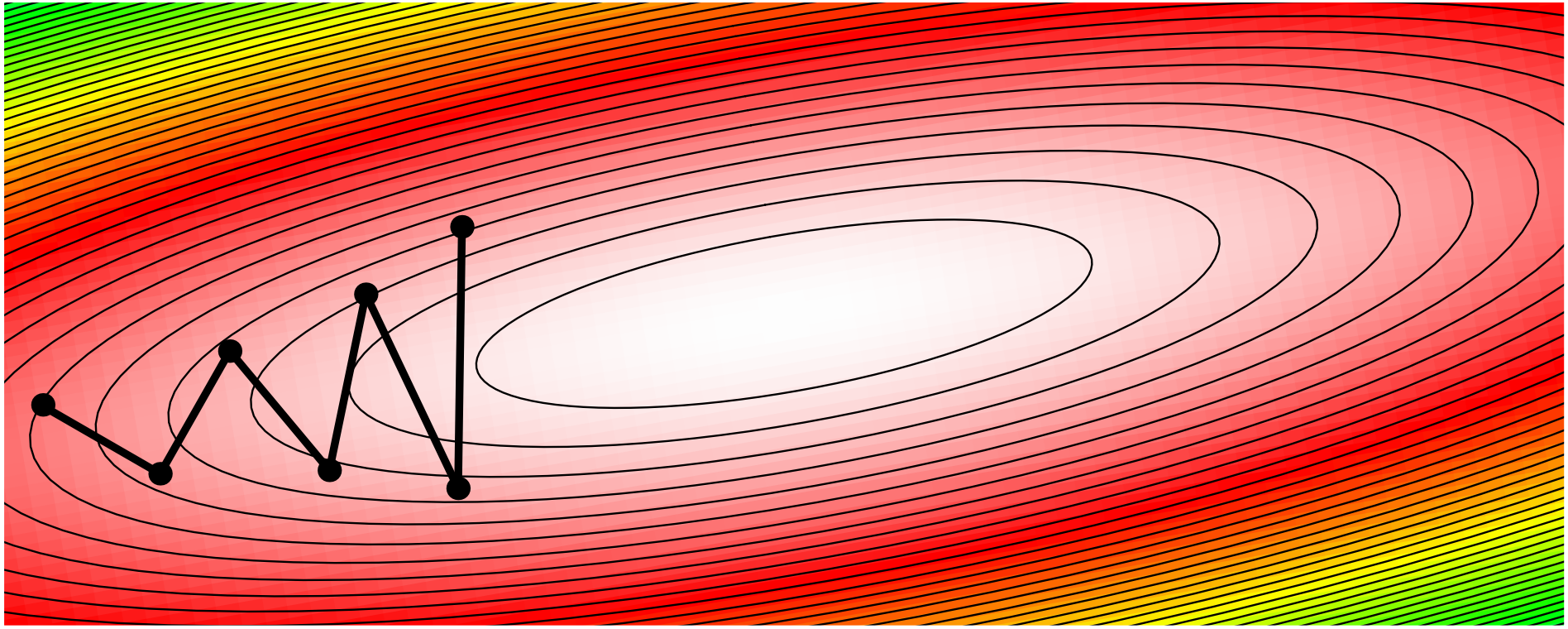
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



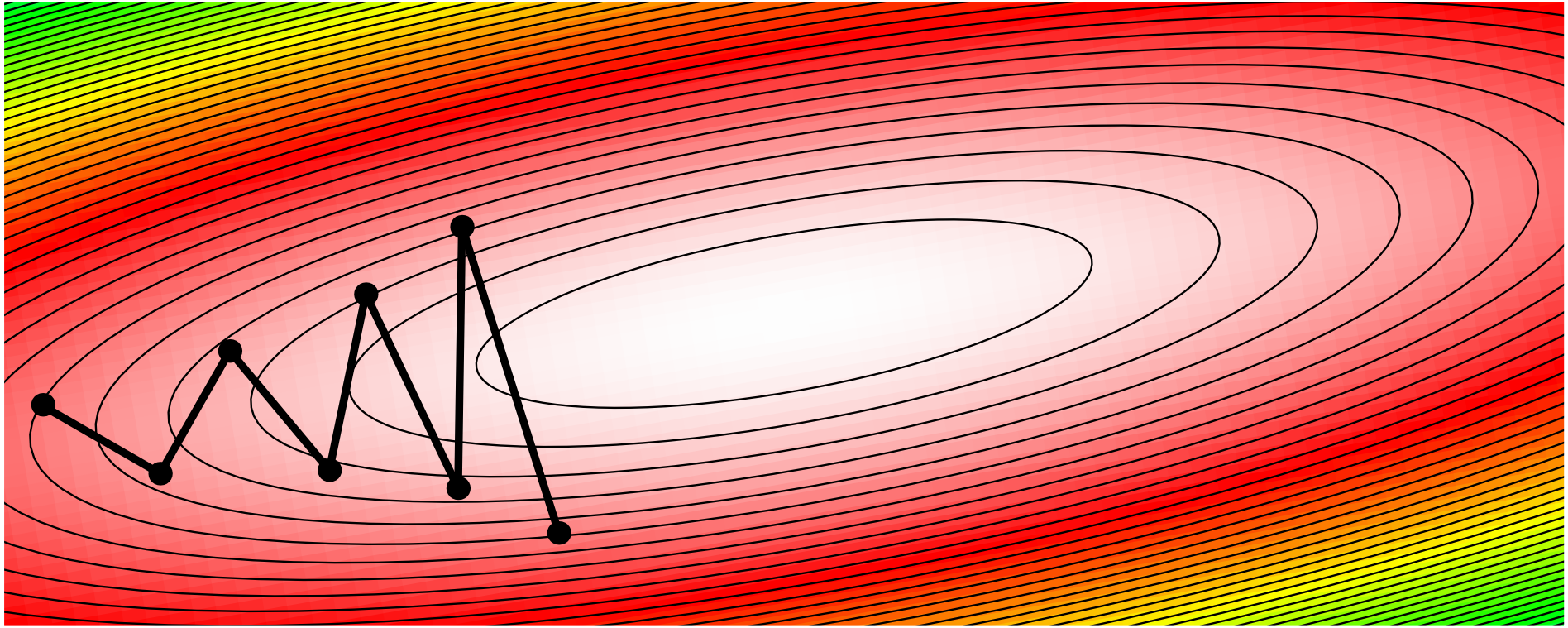
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



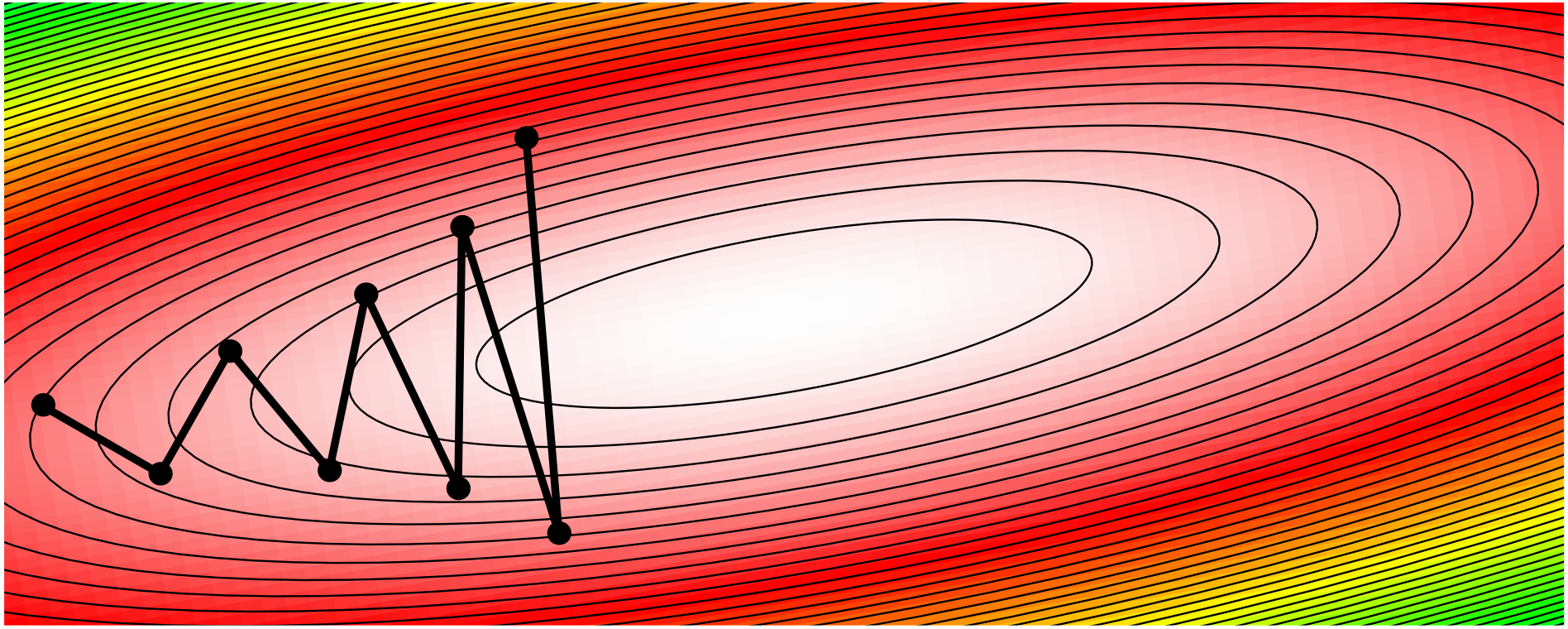
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



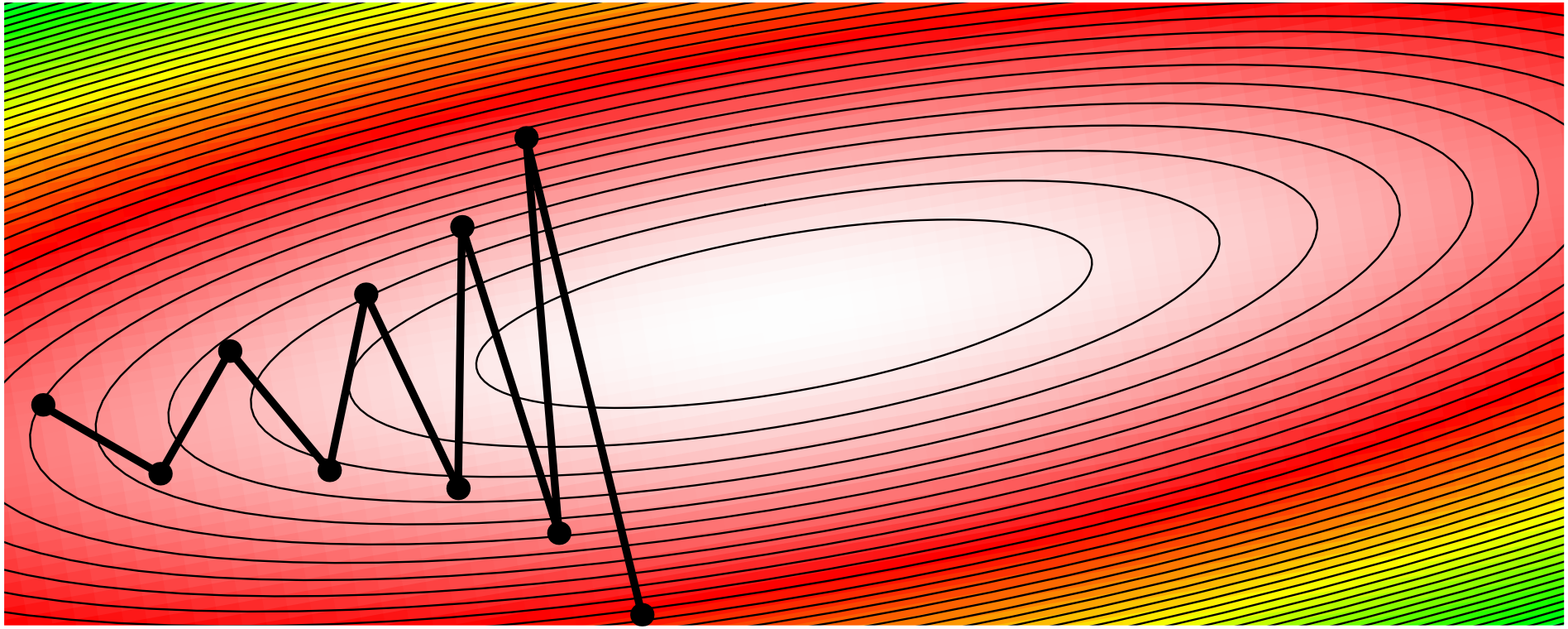
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



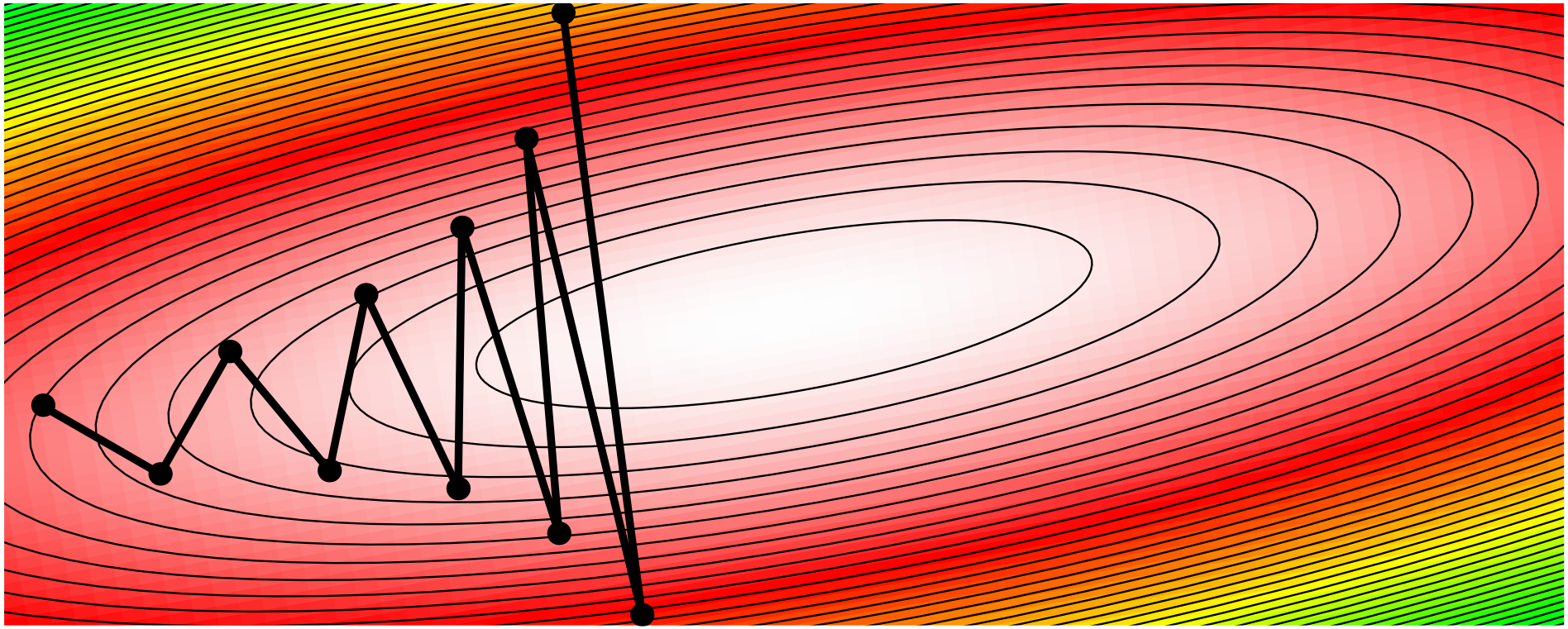
More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



More Haste Less Speed

- Increasing the step size a little increases the rate of converge in one direction, but . . .



Speeding Up Stochastic Gradient Descent

- To speed up stochastic gradient descent we can add a momentum term

$$\boldsymbol{v}(t) = (1-\epsilon)\boldsymbol{v}(t) - r\boldsymbol{\nabla} L(\boldsymbol{\theta}(t-1))$$
$$\boldsymbol{\theta}(t) = \boldsymbol{\theta}(t-1) + \boldsymbol{v}(t)$$

- Many variations on this, e.g. ada-delta and adam

- These are quite new techniques developed for deep-learning, but equally applicable to MLPs

Speeding Up Stochastic Gradient Descent

- To speed up stochastic gradient descent we can add a momentum term

$$\boldsymbol{v}(t) = (1-\epsilon)\boldsymbol{v}(t) - r\boldsymbol{\nabla} L(\boldsymbol{\theta}(t-1))$$
$$\boldsymbol{\theta}(t) = \boldsymbol{\theta}(t-1) + \boldsymbol{v}(t)$$

- Many variations on this, e.g. ada-delta and adam

- These are quite new techniques developed for deep-learning, but equally applicable to MLPs

Speeding Up Stochastic Gradient Descent

- To speed up stochastic gradient descent we can add a momentum term

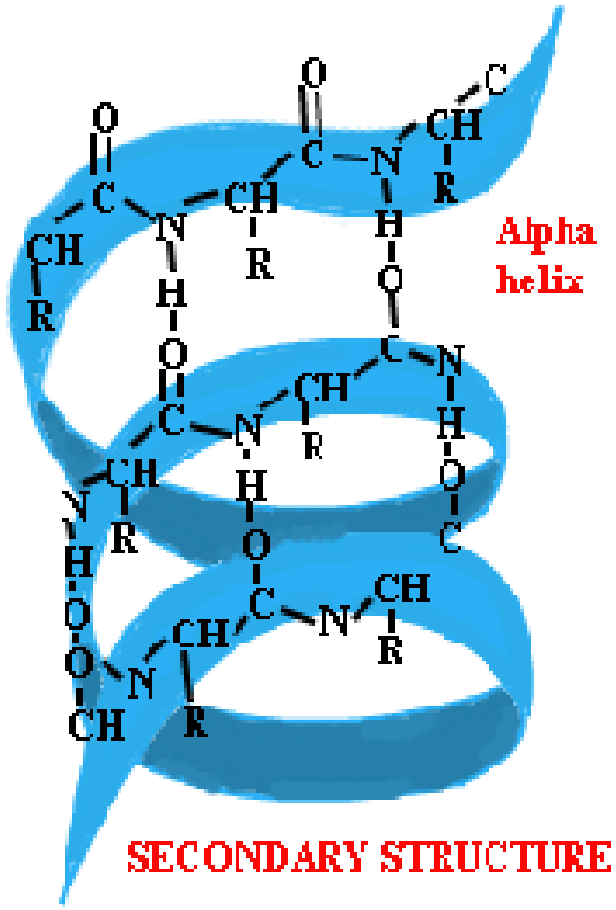
$$\boldsymbol{v}(t) = (1-\epsilon)\boldsymbol{v}(t) - r\boldsymbol{\nabla} L(\boldsymbol{\theta}(t-1))$$
$$\boldsymbol{\theta}(t) = \boldsymbol{\theta}(t-1) + \boldsymbol{v}(t)$$

- Many variations on this, e.g. ada-delta and adam

- These are quite new techniques developed for deep-learning, but equally applicable to MLPs

Outline

1. MLP
2. **Secondary Protein Structure**
3. Designing MLPs



Protein Structure

- Understanding the 3D structure of proteins is very important to understand its function
- Unfortunately, it is extremely difficult to infer its 3D structure
  - ⋆ It is both experimentally very costly
  - ⋆ And computationally very challenging
- However knowing a proteins **primary** structure (sequence) is quite easy (we can read this off if we know the RNA sequence from which it is translated)

Protein Structure

- Understanding the 3D structure of proteins is very important to understand its function
- Unfortunately, it is extremely difficult to infer its 3D structure
  - ⋆ It is both experimentally very costly
  - ⋆ And computationally very challenging
- However knowing a proteins **primary** structure (sequence) is quite easy (we can read this off if we know the RNA sequence from which it is translated)

Protein Structure

- Understanding the 3D structure of proteins is very important to understand its function
- Unfortunately, it is extremely difficult to infer its 3D structure
  - ⋆ It is both experimentally very costly
  - ⋆ And computationally very challenging
- However knowing a proteins **primary** structure (sequence) is quite easy (we can read this off if we know the RNA sequence from which it is translated)

Protein Structure

- Understanding the 3D structure of proteins is very important to understand its function
- Unfortunately, it is extremely difficult to infer its 3D structure
  - ⋆ It is both experimentally very costly
  - ⋆ And computationally very challenging
- However knowing a proteins **primary** structure (sequence) is quite easy (we can read this off if we know the RNA sequence from which it is translated)

Protein Structure

- Understanding the 3D structure of proteins is very important to understand its function
- Unfortunately, it is extremely difficult to infer its 3D structure
  - ⋆ It is both experimentally very costly
  - ⋆ And computationally very challenging
- However knowing a proteins **primary** structure (sequence) is quite easy (we can read this off if we know the RNA sequence from which it is translated)

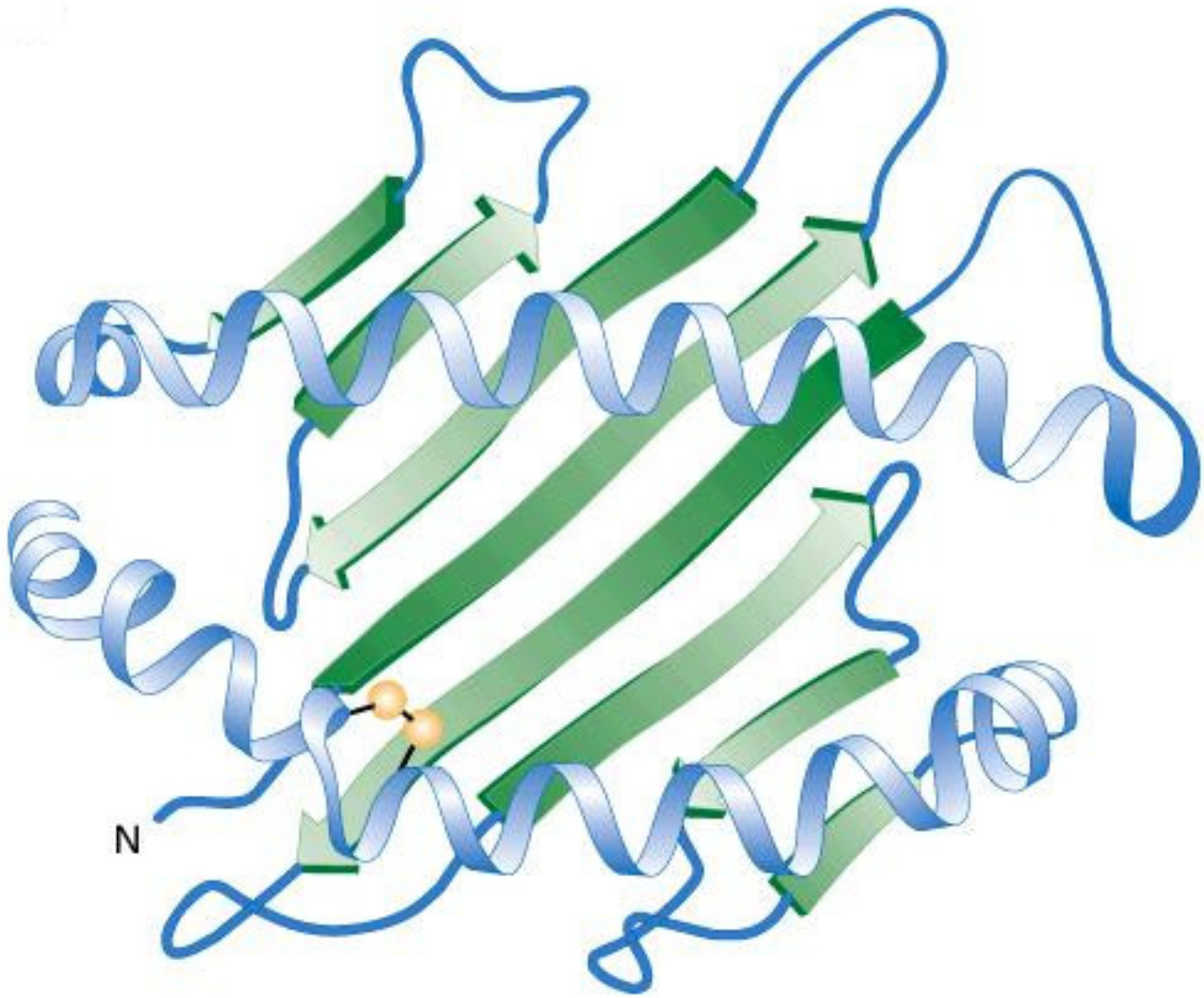
Secondary Structure

- The secondary structure tells us whether a protein is part of
  - ⋆ An alpha-helix
  - ⋆ A beta-sheet
  - ⋆ Something else (often referred to as coil)
- This does not tell us the full 3-D structure but it is the first step towards inferring this

Secondary Structure

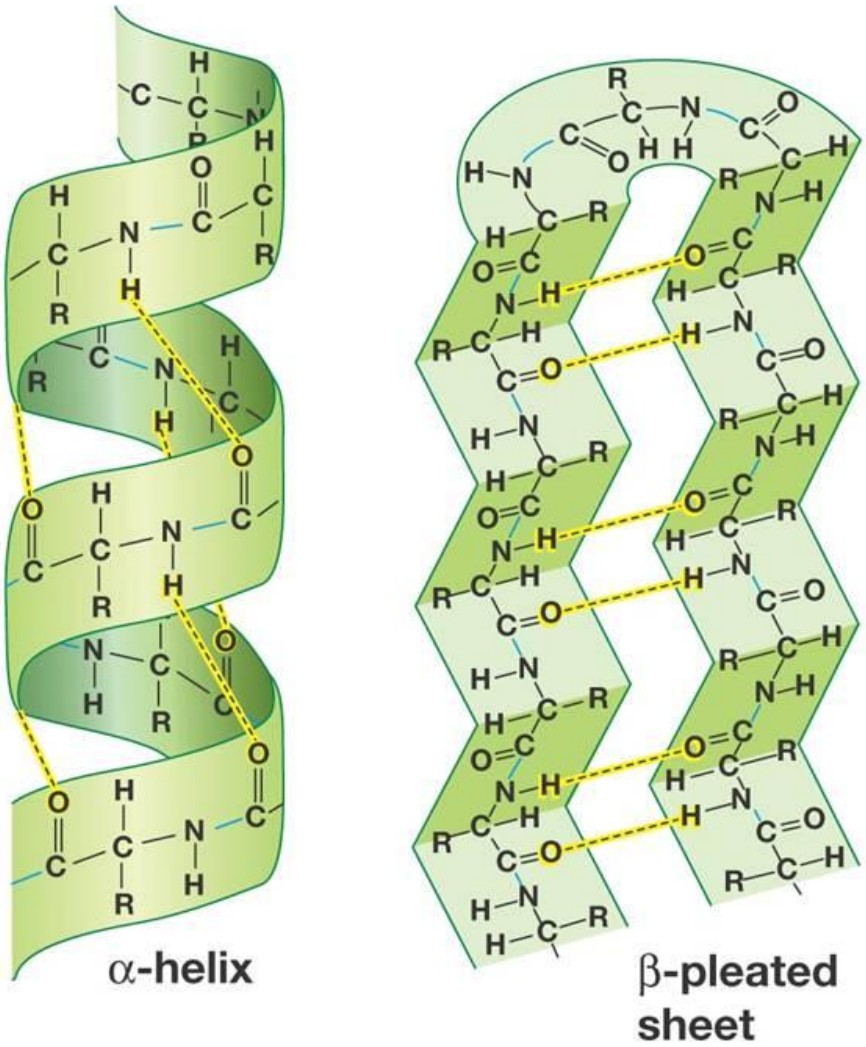
- The secondary structure tells us whether a protein is part of
  - ⋆ An alpha-helix
  - ⋆ A beta-sheet
  - ⋆ Something else (often referred to as coil)
- This does not tell us the full 3-D structure but it is the first step towards inferring this

Protein Structure



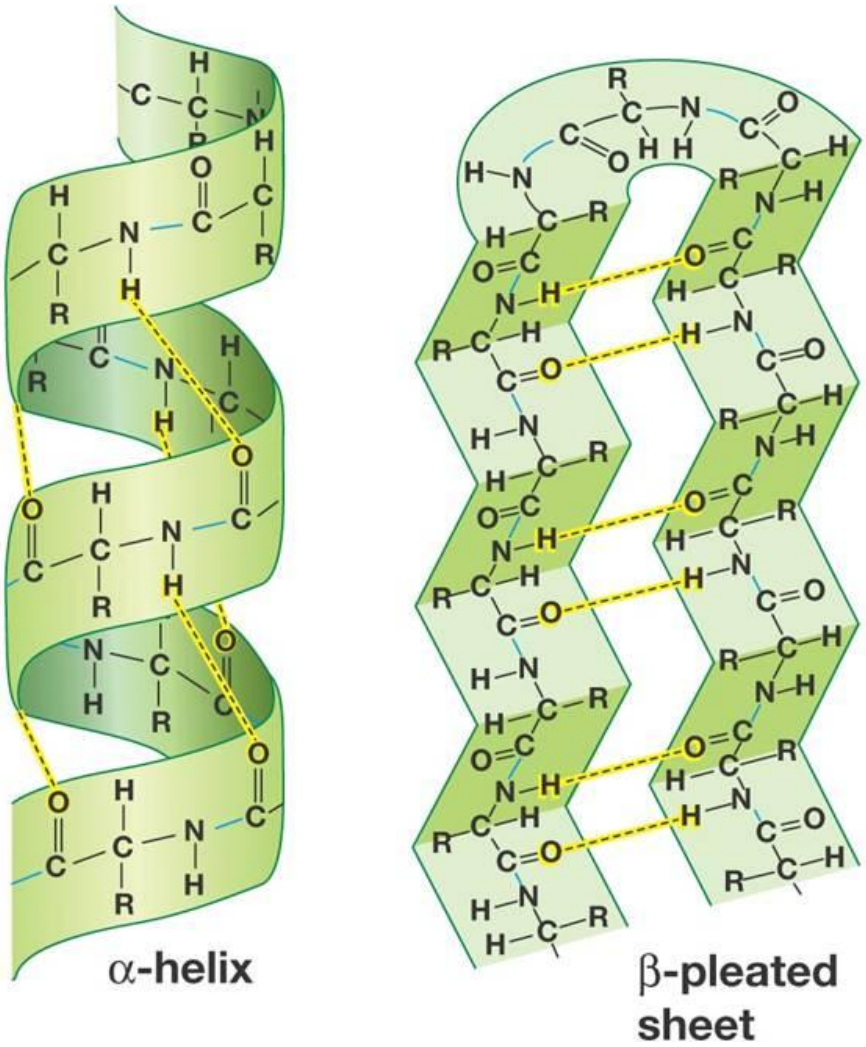
Test Problem

- Secondary protein structure has long been a classic test problem for machine learning
- It was an early success for MLPs
- People have tried SVMs, HMMs, etc.
- It is a difficult problem in part because it is non-local



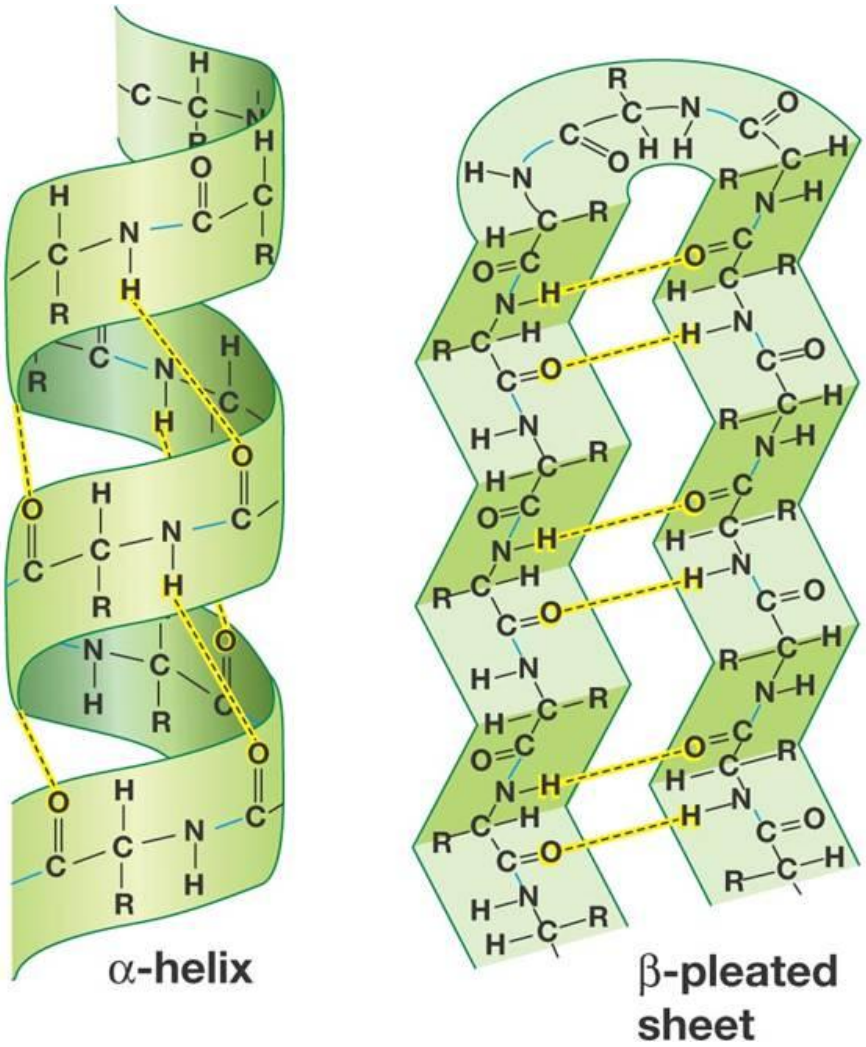
Test Problem

- Secondary protein structure has long been a classic test problem for machine learning
- It was an early success for MLPs
- People have tried SVMs, HMMs, etc.
- It is a difficult problem in part because it is non-local



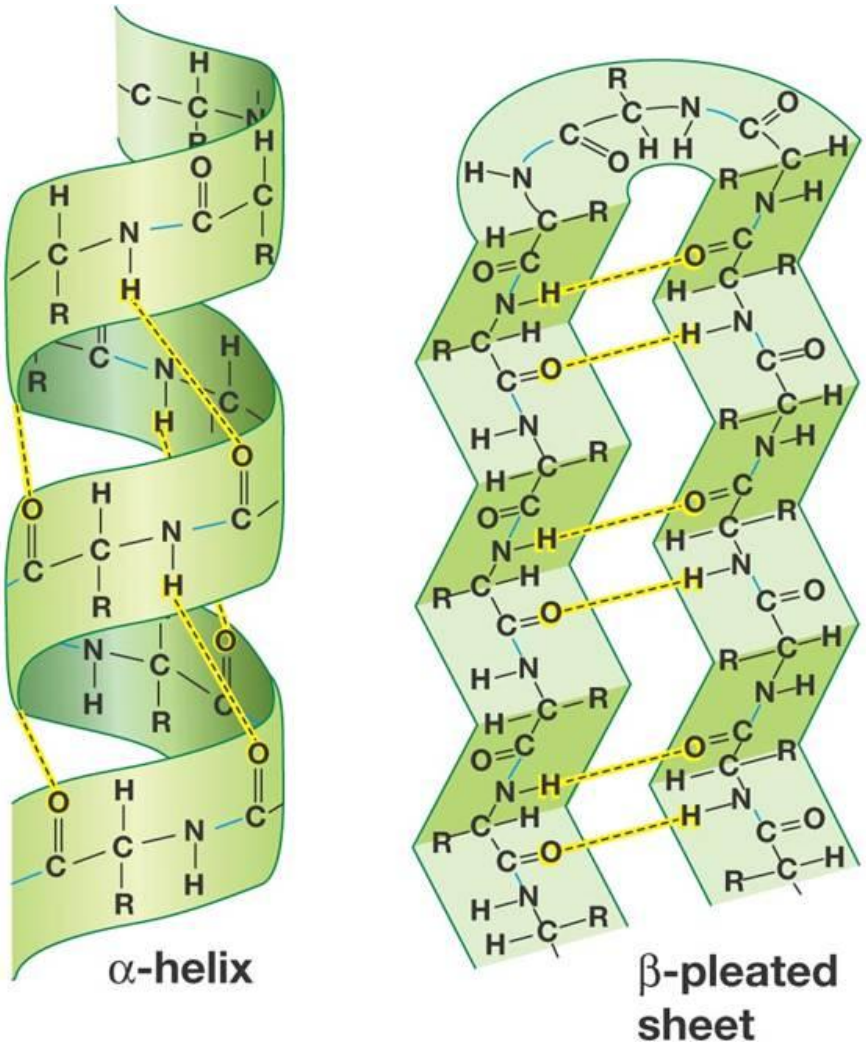
Test Problem

- Secondary protein structure has long been a classic test problem for machine learning
- It was an early success for MLPs
- People have tried SVMs, HMMs, etc.
- It is a difficult problem in part because it is non-local



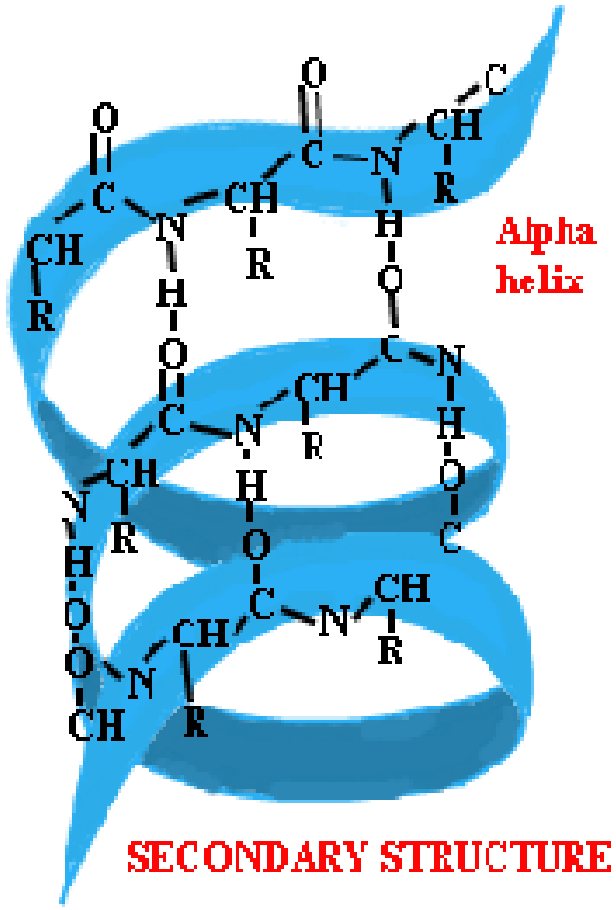
Test Problem

- Secondary protein structure has long been a classic test problem for machine learning
- It was an early success for MLPs
- People have tried SVMs, HMMs, etc.
- It is a difficult problem in part because it is non-local



Outline

1. MLP
2. Secondary Protein Structure
3. **Designing MLPs**



Predicting Secondary Structure

- Although very costly we know the secondary structure for quite a few proteins
- This provides a training and testing data set
- However, its not immediately obvious how to create a neural network to use this data
- Most learning machines take a fixed length input

Predicting Secondary Structure

- Although very costly we know the secondary structure for quite a few proteins
- This provides a training and testing data set
- However, its not immediately obvious how to create a neural network to use this data
- Most learning machines take a fixed length input

Predicting Secondary Structure

- Although very costly we know the secondary structure for quite a few proteins
- This provides a training and testing data set
- However, its not immediately obvious how to create a neural network to use this data
- Most learning machines take a fixed length input

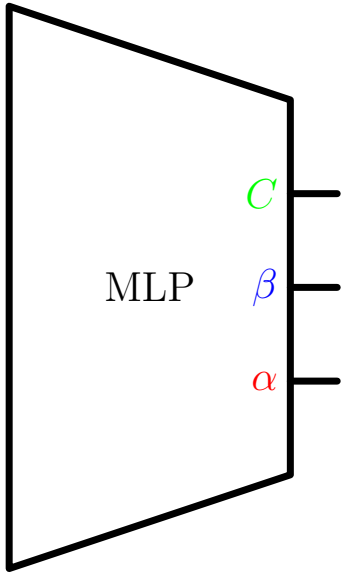
Predicting Secondary Structure

- Although very costly we know the secondary structure for quite a few proteins
- This provides a training and testing data set
- However, its not immediately obvious how to create a neural network to use this data
- Most learning machines take a fixed length input

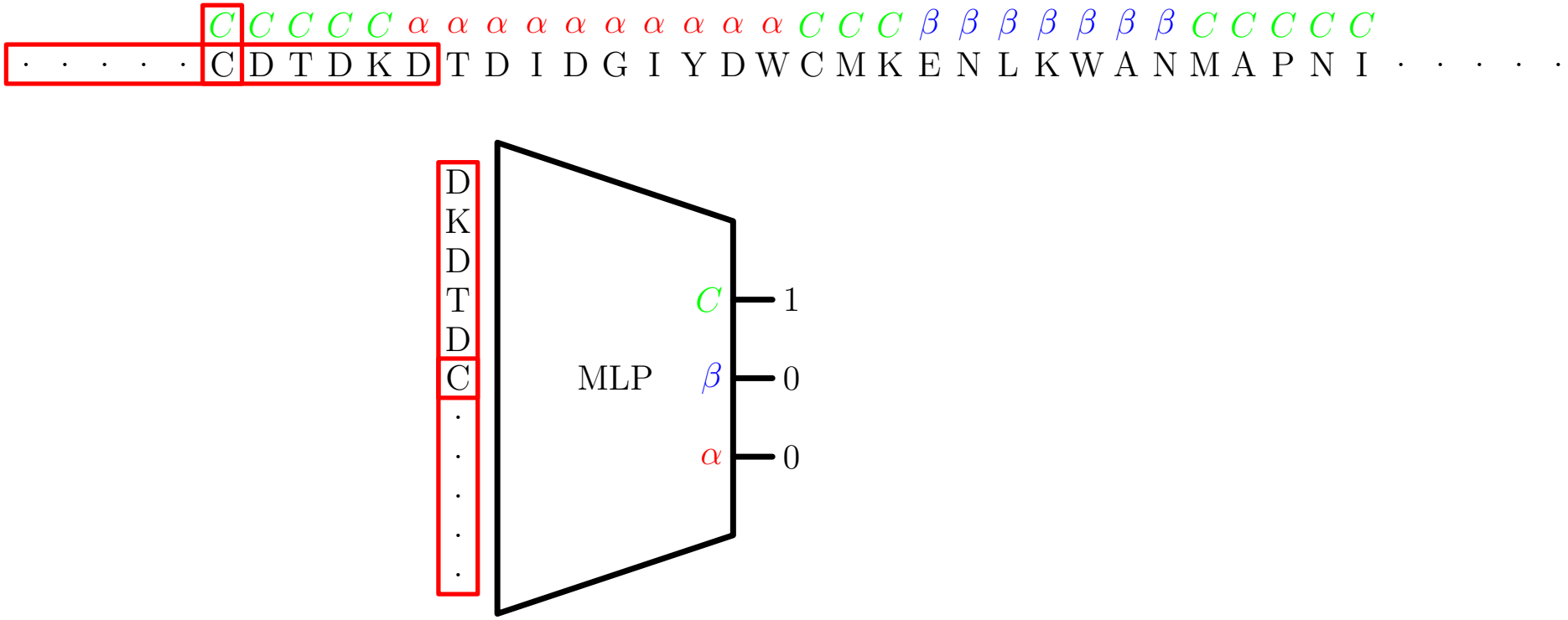
Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

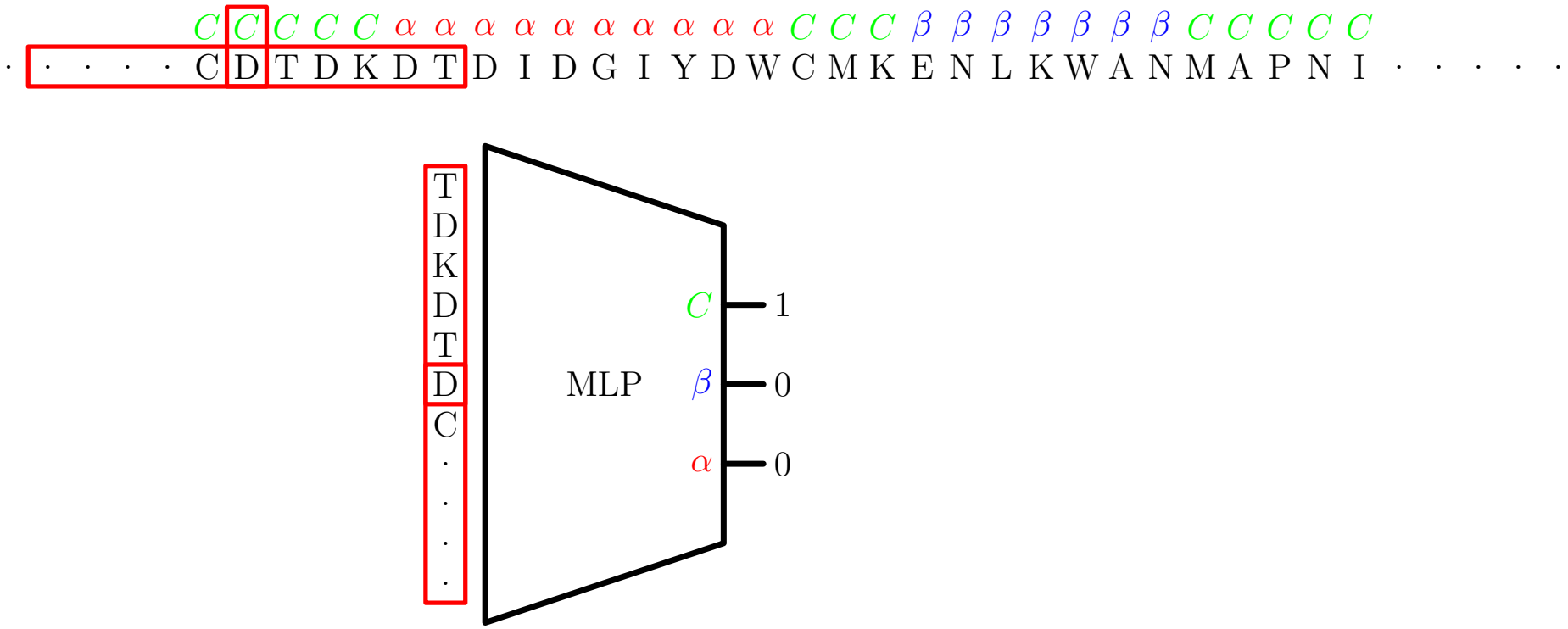
· · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · ·



Windowing


C C C C C α α α α α α α α α α C C C β β β β β β β C C C C C
· · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · ·
D
K
D
T
D
C
MLP
C
β
α
1
0
0

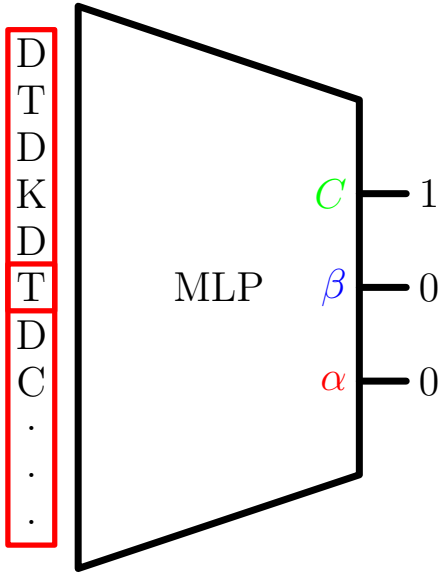
Windowing


C C C C C α α α α α α α α α α C C C β β β β β β β C C C C C
· · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · · ·
T
D
K
D
T
D
C
·
·
·
·
MLP
C
β
α
1
0
0

Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

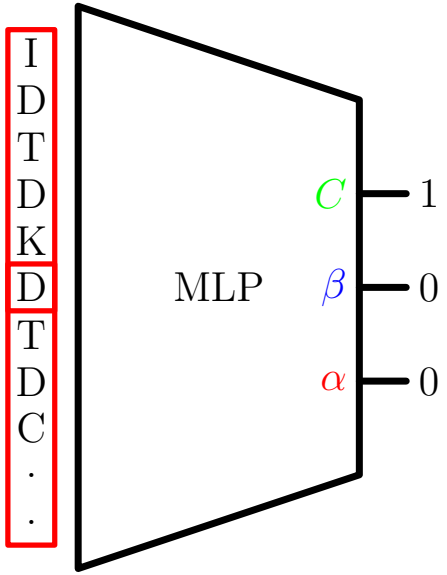
· · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · · ·



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

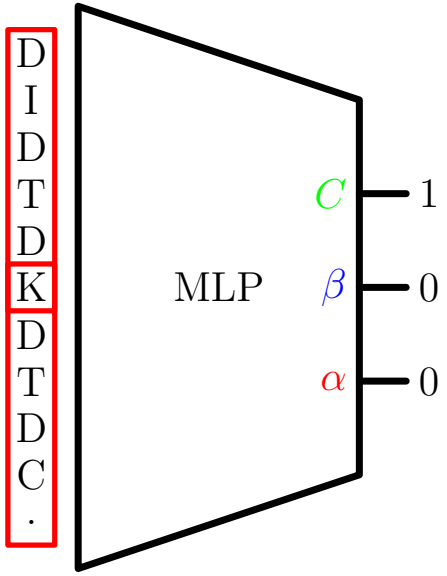
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

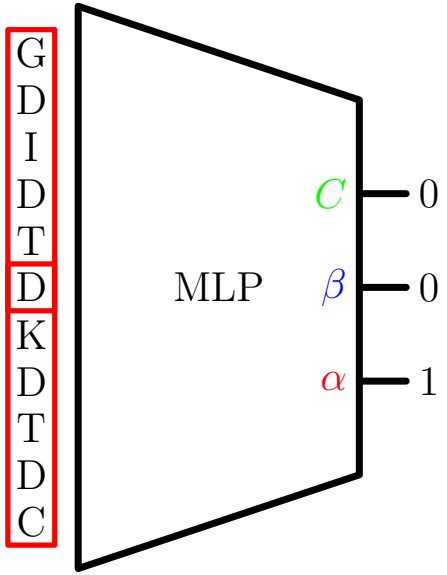
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

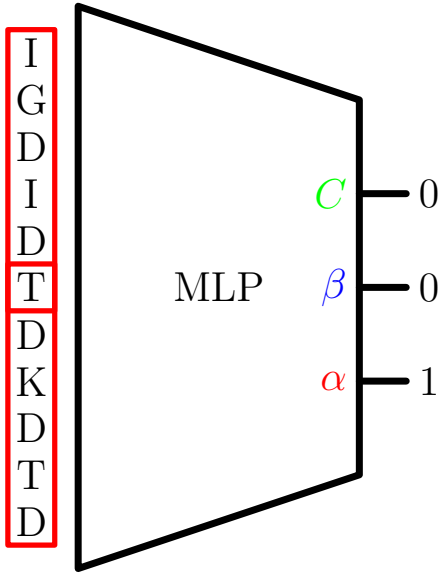
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

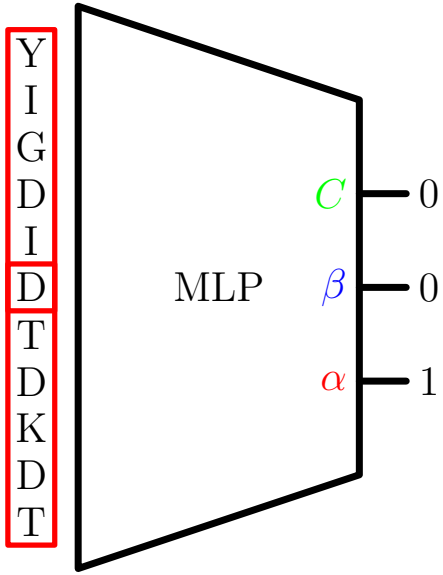
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

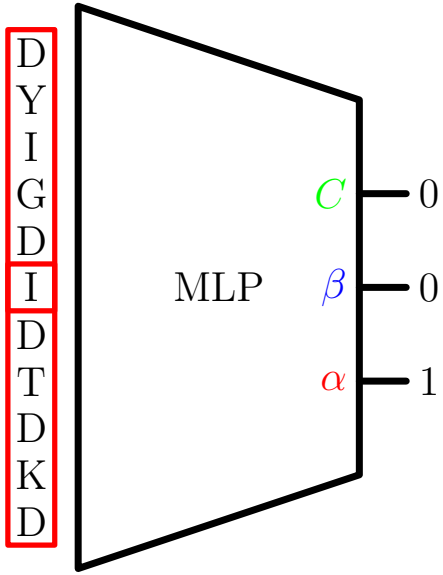
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

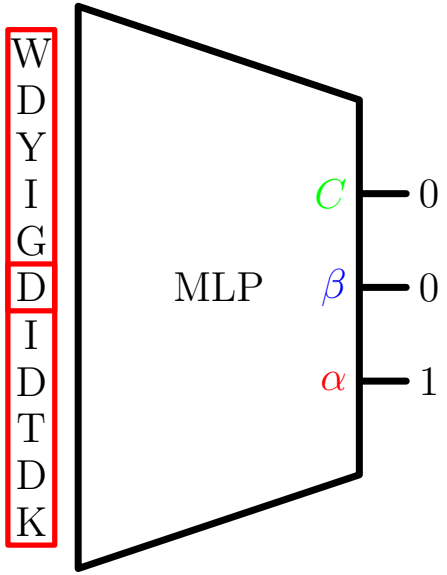
. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Windowing

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .

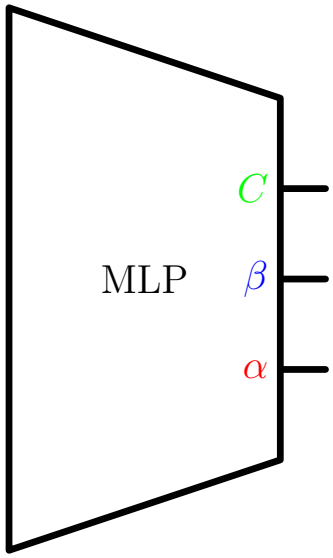


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

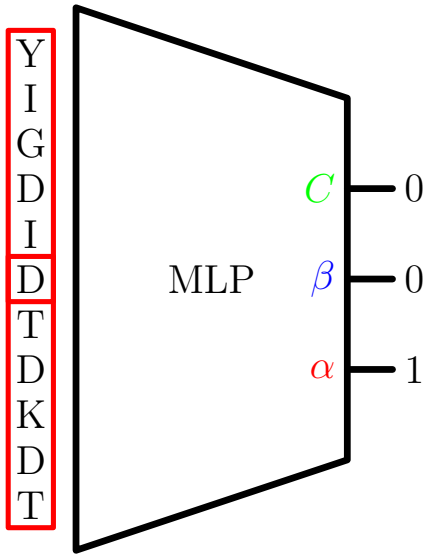
$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

· · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · · ·



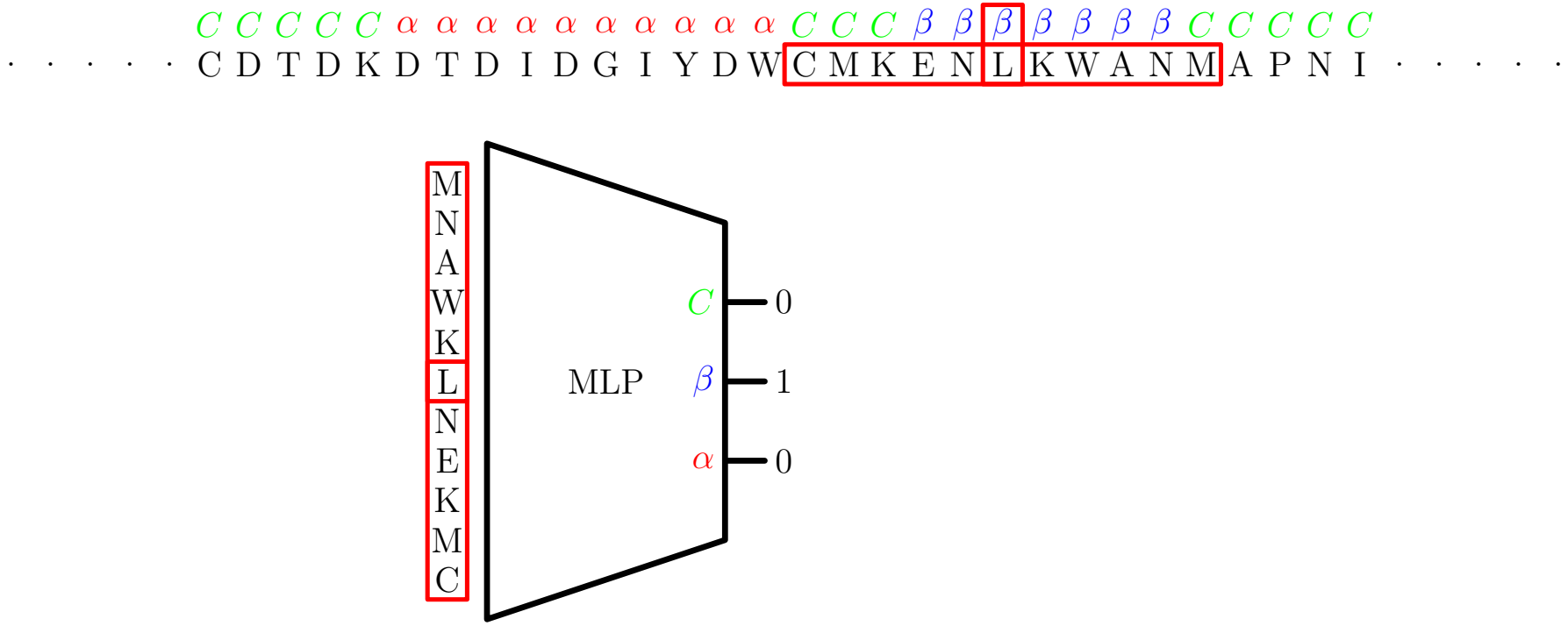
Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure



Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

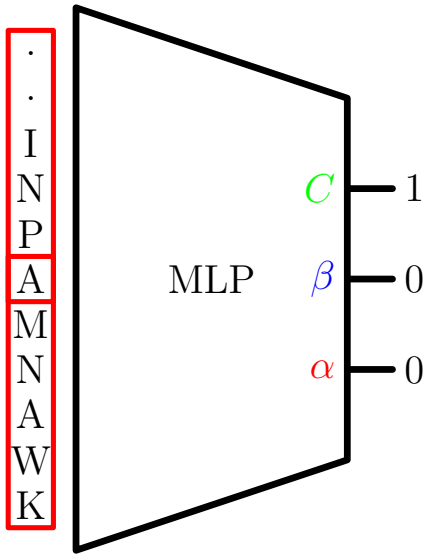


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C\ C$

· · · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · · ·

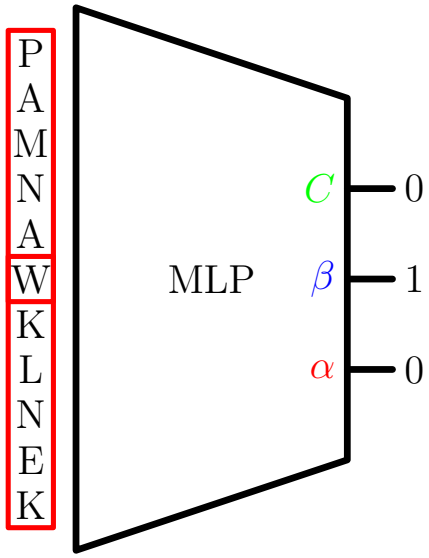


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .

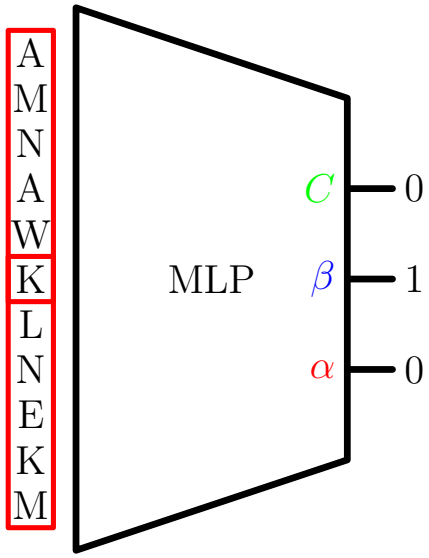


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

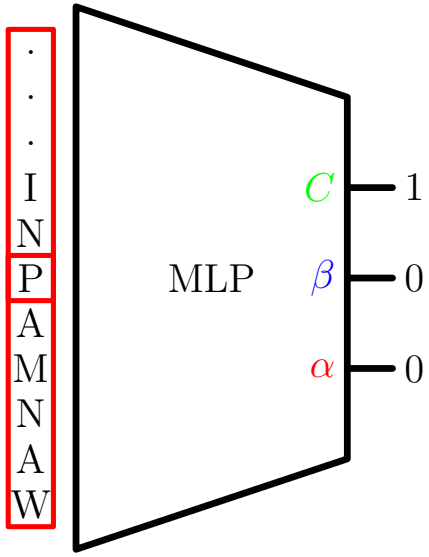
$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



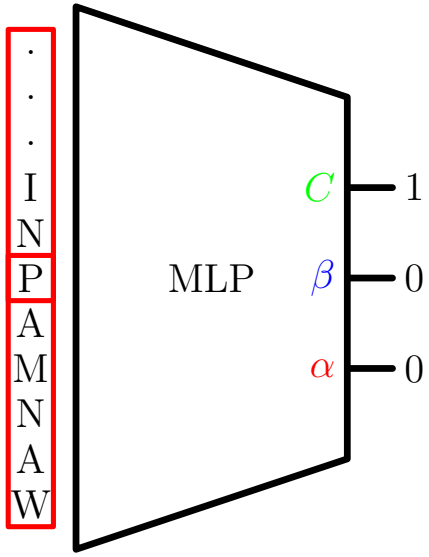
Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure



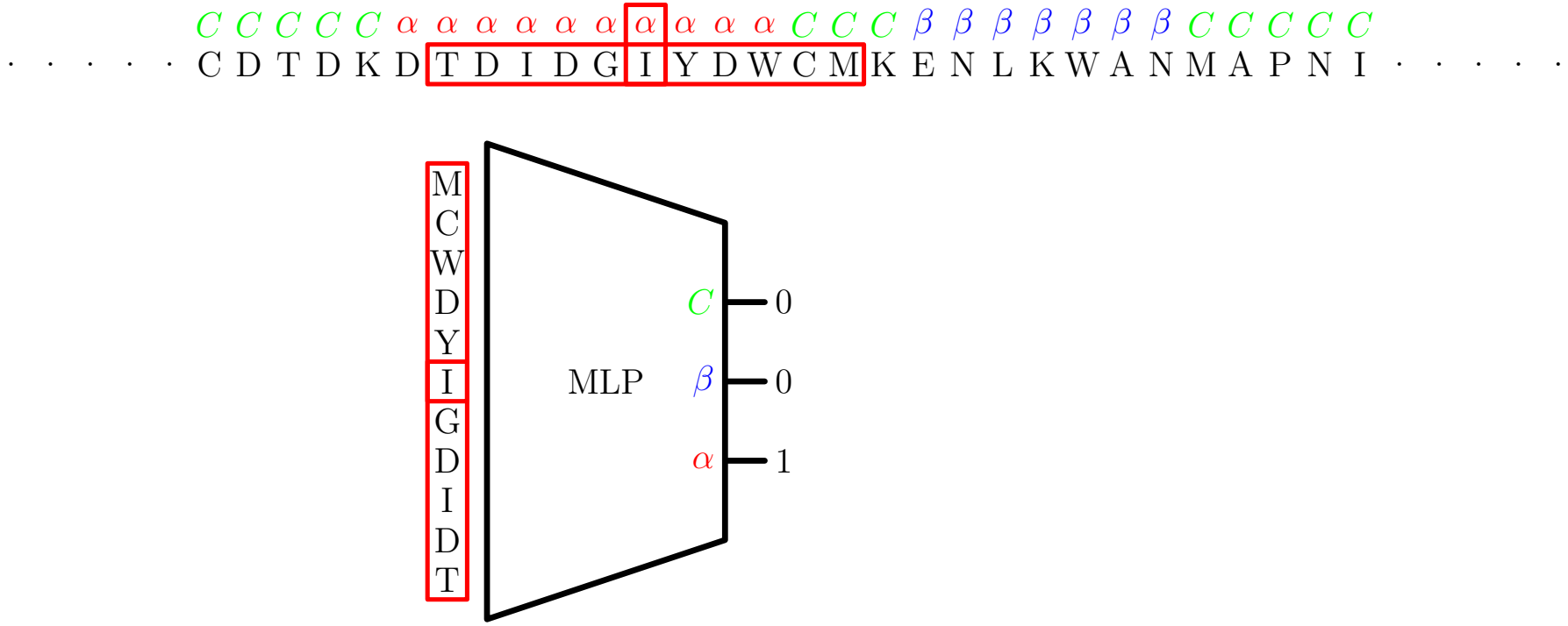
Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure



Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

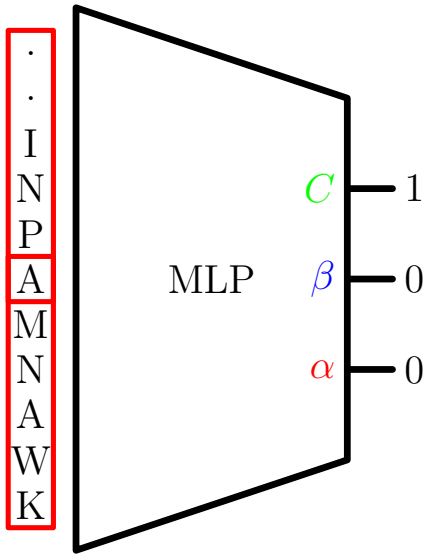


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C\ C$

· · · · · · C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I · · · · · ·

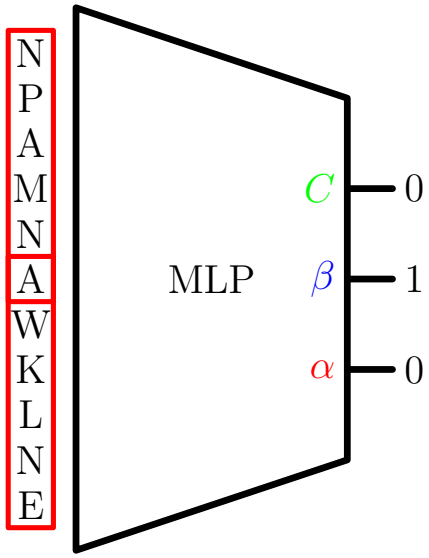


Randomised Windows

- During training we can randomise the placement of the window so we don't learn a long section of one structure

$C\ C\ C\ C\ C\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ \alpha\ C\ C\ C\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ \beta\ C\ C\ C\ C\ C$

. . . . . C D T D K D T D I D G I Y D W C M K E N L K W A N M A P N I . . . . . .



Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Representing Proteins

- How should we feed our protein to an MLP?
- Providing a number from 1–20 would put in an artificial ordering
- We could try and put in chemical properties, e.g. acid/base hydophobic/hydophilic, etc. but which ones
- It turned out best to use a **1-hot** (orthogonal) representation

$$\begin{aligned}
\text{Glycine (G)} &= (1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
\text{Alanine (A)} &= (0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \\
&\vdots \\
\text{End} &= (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1)
\end{aligned}$$

- This is unbiased and seems to work best

Output

- Usually have 3 output (alpha-helix, beta-sheet and coil) although sometimes coil is split into 6 classes
- Again use a 1-hot representation
- Use winner-takes-all prediction (largest output)

Output

- Usually have 3 output (alpha-helix, beta-sheet and coil) although sometimes coil is split into 6 classes
- Again use a 1-hot representation
- Use winner-takes-all prediction (largest output)

Output

- Usually have 3 output (alpha-helix, beta-sheet and coil) although sometimes coil is split into 6 classes
- Again use a 1-hot representation
- Use winner-takes-all prediction (largest output)

Softmax

- A slightly better network is to use a **softmax** output where

$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = \frac{\mathrm{e}^{\sum_{j=1}^{h} \bar{W}_{ij} h_j(\boldsymbol{x}) + \bar{b}_i}}{\sum_{k=1}^{3} \mathrm{e}^{\sum_{j=1}^{h} \bar{W}_{kj} h_j(\boldsymbol{x}) + \bar{b}_i}}$$

- where $h_j(\boldsymbol{x})$ is the output of the jth hidden unit

- Note that we can the interpret the output as

$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = \mathbb{P}\left(\text{class } i|\boldsymbol{x},\, \boldsymbol{\theta}\right)$$

Softmax

- A slightly better network is to use a **softmax** output where

$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = \frac{\mathrm{e}^{\sum_{j=1}^{h} \bar{W}_{ij} h_j(\boldsymbol{x}) + \bar{b}_i}}{\sum_{k=1}^{3} \mathrm{e}^{\sum_{j=1}^{h} \bar{W}_{kj} h_j(\boldsymbol{x}) + \bar{b}_i}}$$

- where $h_j(\boldsymbol{x})$ is the output of the jth hidden unit

- Note that we can the interpret the output as

$$f_i(\boldsymbol{x}|\boldsymbol{\theta}) = \mathbb{P}\left(\text{class } i|\boldsymbol{x},\, \boldsymbol{\theta}\right)$$

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Relative Entropy

- Given a training set $\mathcal{D} = \big((\boldsymbol{x}^\mu, y^\mu) \,\big|\, \mu = 1, 2, \ldots, P\big)$ where $y^\mu \in \mathcal{S} = \{\alpha\text{-helix}, \beta\text{-sheet}, \text{Coil}\}$, the log-likelihood of the data is

$$L(\boldsymbol{\theta}) = \log(\mathbb{P}\,(\mathcal{D}|\boldsymbol{\theta})) = \sum_{\mu=1}^{P} \log(\mathbb{P}\,(y^\mu|\boldsymbol{x}^\mu, \boldsymbol{\theta}))$$

$$= \sum_{\mu=1}^{P} \log(f_{y^\mu}(\boldsymbol{x}^\mu|\boldsymbol{\theta}))$$

- This is known as the **cross entropy**

- Rather than minimising the squared error we can maximise the relative entropy—again we use a gradient based technique

Simple ANN

- A simple network would consist of
  - $W \times 21$ inputs where W is the window size (typically around 11)
  - n_h hidden sigmoidal units (typically around 50)
  - 3 softmax output units
- Results are around 60% prediction of the three classes

Simple ANN

- A simple network would consist of
  - ⋆ $W \times 21$ inputs where W is the window size (typically around 11)
  - ⋆ n_h hidden sigmoidal units (typically around 50)
  - ⋆ 3 softmax output units
- Results are around 60% prediction of the three classes

Simple ANN

- A simple network would consist of
  - ⋆ $W \times 21$ inputs where W is the window size (typically around 11)
  - ⋆ n_h hidden sigmoidal units (typically around 50)
  - ⋆ 3 softmax output units
- Results are around 60% prediction of the three classes

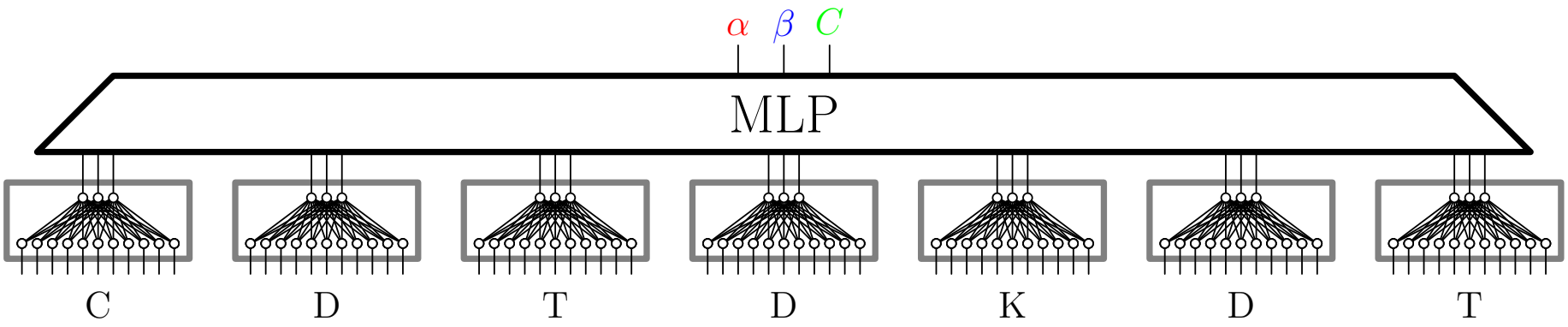
Simple ANN

- A simple network would consist of
  - ⋆ $W \times 21$ inputs where W is the window size (typically around 11)
  - ⋆ n_h hidden sigmoidal units (typically around 50)
  - ⋆ 3 softmax output units
- Results are around 60% prediction of the three classes

Learning A Better Input Representation

- The 1-hot vector representation of the input makes the network very large

- We can learn an encoding or **embedding** by using the same 21 to M layer for all inputs in the window



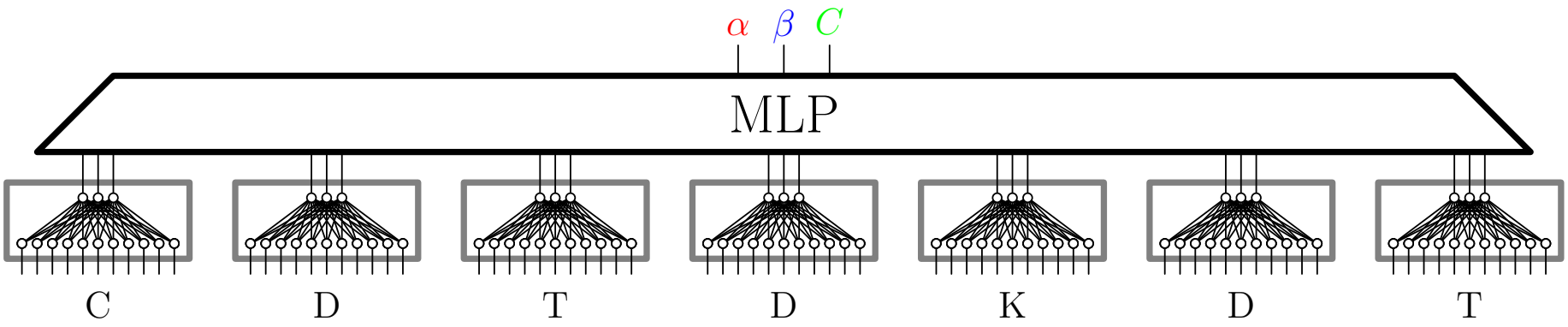
- All the input encoding networks share the same weights

- This slightly complicates back-prop, but is not too difficult

Learning A Better Input Representation

- The 1-hot vector representation of the input makes the network very large

- We can learn an encoding or **embedding** by using the same 21 to M layer for all inputs in the window



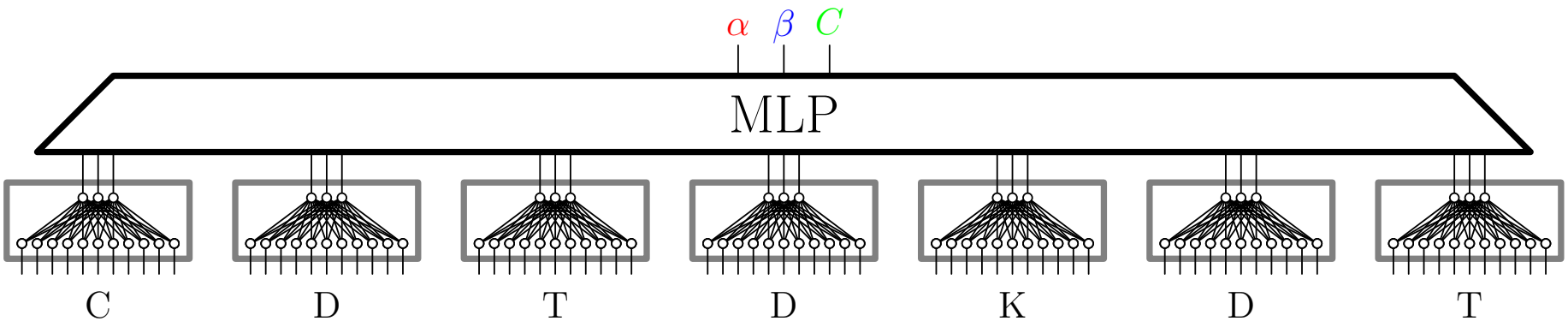
- All the input encoding networks share the same weights

- This slightly complicates back-prop, but is not too difficult

Learning A Better Input Representation

- The 1-hot vector representation of the input makes the network very large

- We can learn an encoding or **embedding** by using the same 21 to M layer for all inputs in the window



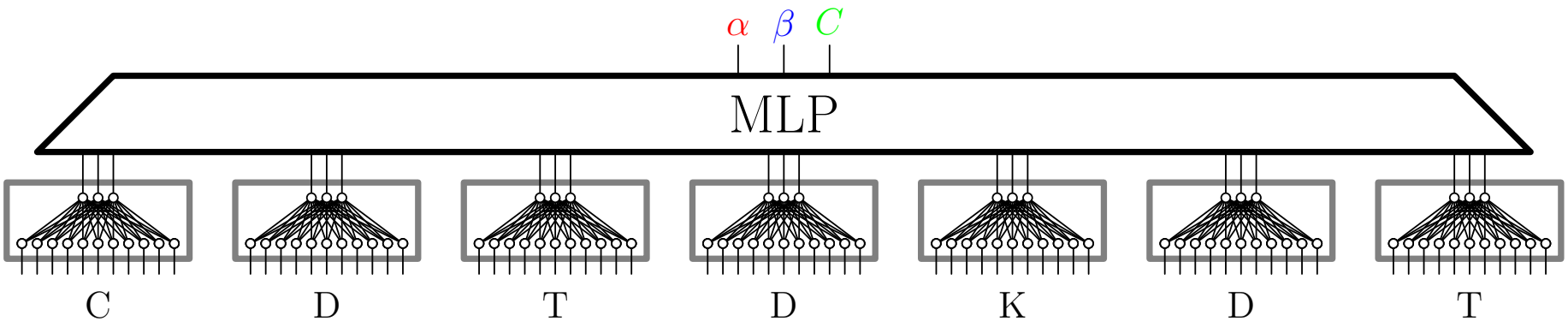
- All the input encoding networks share the same weights

- This slightly complicates back-prop, but is not too difficult

Learning A Better Input Representation

- The 1-hot vector representation of the input makes the network very large

- We can learn an encoding or **embedding** by using the same 21 to M layer for all inputs in the window



- All the input encoding networks share the same weights

- This slightly complicates back-prop, but is not too difficult

Top Tier

- When testing we get a prediction at each site of the sequence
- We can improve our machine by building a second machine that takes the prediction at a site and its neighbours
- This uses slightly more global information and prevents silly predictions, e.g. α, β, α, C
- In machine learning this is known as stacking

Top Tier

- When testing we get a prediction at each site of the sequence
- We can improve our machine by building a second machine that takes the prediction at a site and its neighbours
- This uses slightly more global information and prevents silly predictions, e.g. α, β, α, C
- In machine learning this is known as stacking

Top Tier

- When testing we get a prediction at each site of the sequence
- We can improve our machine by building a second machine that takes the prediction at a site and its neighbours
- This uses slightly more global information and prevents silly predictions, e.g. α, β, α, C
- In machine learning this is known as stacking

Top Tier

- When testing we get a prediction at each site of the sequence
- We can improve our machine by building a second machine that takes the prediction at a site and its neighbours
- This uses slightly more global information and prevents silly predictions, e.g. α, β, α, C
- In machine learning this is known as stacking

Using Profiles

- The big improvement in prediction came from using the protein profile obtained by finding similar protein structures

```
A5ASC3.1    14 SIKLWPPSQTTRLLLVERMANNLST..PSIFTRK..YGSLSKEEARENAKQIEEVACSTANQ.....HYEKEPDGDGGSAVQLYAKECSKLILEVLK 101
B4F917.1    13 SIKLWPPSESTRIMLVDRMTNNLST..ESIFSRK..YRLLGKQEAHENAKTIEELCFALADE.....HFREEPDGDGSSAVQLYAKETSKMMLEVLK 100
A9S1V2.1    23 VFKLWPPSQGTREAVRQKMALKLSS..ACFESQS..FARIELADAQEHARAIEEVAFGAAQE......ADSGGDKTGSAVVMVYAKHASKLMLETLR 109
B9GSN7.1    13 SVKLWPPGQSTRLMLVERMTKNFIT..PSFISRK..YGLLSKEEAEEDAKKIEEVAFAAANQ.....HYEKQPDGDGSSAVQIYAKESSRLMLEVLK 100
Q8H056.1    30 SFSIWPPTQRTRDAVVRRLVDTLGG..DTILCKR..YGAVPAADAEPAARGIEAEAFDAAAA..SGEAAATASVEEGIKALQLYSKEVSRRLLDFVK 120
Q0D4Z3.2    44 SLSIWPPSQRTRDAVVRRLVQTLVA..PSILSQR..YGAVPEAEAGRAAAAVEAEAYAAVTES.SSAAAAPASVEDGIEVLQAYSKEVSRRLLELAK 135
B9MVW8.1    56 SFSIWPPTQRTRDAIISRLIETLST..TSVLSKR..YGTIPKEEASEASRRIEEEAFSGAST.......VASSEKDGLEVLQLYSKEISKRMLETVK 141
Q0IYC5.1    29 SFAVWPPTRRTRDAVVRRLVAVLSGDTTTALRKRYRYGAVPAADAERAARAVEAQAFDAASA....SSSSSSSVEDGIETLQLYSREVSNRLLAFVR 121
A9NW46.1    13 SIKLWPPSESTRLMLVERMTDNLSS..VSFFSRK..YGLLSKEEAAENAKRIEETAFLAAND.....HEAKEPNLDDSSVVQFYAREASKLMLEALK 100
Q9C500.1    57 SLRIWPPTQKTRDAVLNRLIETLST..ESILSKR..YGTLKSDDATTVAKLIEEEAYGVASN.......AVSSDDDGIKILELYSKEISKRMLESVK 142
Q2HRI7.1    25 MYSIWPPKQRTRDAVKNRLIETLST..PSVLTKR..YGTMSADEASAAAIQIEDEAFSVANA.......SSSTSNDNVTILEVYSKEISKRMIETVK 110
Q9M7N3.1    28 SFKIWPPTQRTREAVVRRLVETLTS..QSVLSKR..YGVIPEEDATSAARIIEEEAFSVASV.ASAASTGGRPEDEWIEVLHIYSQEIXQRVVESAK 119
Q9M7N6.1    25 SFSIWPPTQRTRDAVINRLIESLST..PSILSKR..YGTLPQDEASETARLIEEEAFAAAGS.......TASDADDGIEILQVYSKEISKRMIDTVK 110
Q9LE82.1    14 SVKMWPPSKSTRLMLVERMTKNITT..PSIFSRK..YGLLSVEEAEQDAKRIEDLAFATANK.....HFQNEPDGDGTSAVHVYAKESSKLMLDVIK 101
Q9M651.2    13 SIKLWPPSLPTRKALIERITNNFSS..KTIFTEK..YGSLTKDQATENAKRIEDIAFSTANQ.....QFEREPDGDGGSAVQLYAKECSKLILEVLK 100
B9R748.1    48 SLSIWPPTQRTRDAVITRLIETLSS..PSVLSKR..YGTISHDEAESAARRIEDEAFGVANT.......ATSAEDDGLEILQLYSKEISRRMLDTVK 133
```

Profile Encoding

- Instead of using the protein sequence as input we use a coding showing all possible substitutions, deletions or insertions at a site
- This works because protein structure is much better conserved than protein sequences
- Thus related protein sequences will with high probability have the same secondary structure
- The protein profiles are "smoothed" by the related sequence information
- This improved performance by $\sim 5\%$

Profile Encoding

- Instead of using the protein sequence as input we use a coding showing all possible substitutions, deletions or insertions at a site
- This works because protein structure is much better conserved than protein sequences
- Thus related protein sequences will with high probability have the same secondary structure
- The protein profiles are "smoothed" by the related sequence information
- This improved performance by $\sim 5\%$

Profile Encoding

- Instead of using the protein sequence as input we use a coding showing all possible substitutions, deletions or insertions at a site
- This works because protein structure is much better conserved than protein sequences
- Thus related protein sequences will with high probability have the same secondary structure
- The protein profiles are "smoothed" by the related sequence information
- This improved performance by $\sim 5\%$

Profile Encoding

- Instead of using the protein sequence as input we use a coding showing all possible substitutions, deletions or insertions at a site
- This works because protein structure is much better conserved than protein sequences
- Thus related protein sequences will with high probability have the same secondary structure
- The protein profiles are "smoothed" by the related sequence information
- This improved performance by $\sim 5\%$

Profile Encoding

- Instead of using the protein sequence as input we use a coding showing all possible substitutions, deletions or insertions at a site
- This works because protein structure is much better conserved than protein sequences
- Thus related protein sequences will with high probability have the same secondary structure
- The protein profiles are "smoothed" by the related sequence information
- This improved performance by $\sim 5\%$

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance

PSIPRED

- Profiling can be taken further. Rather than just finding matching sequences we can iterate the process
- We use the matched sequences to find other related sequences
- This is what Psi-Blast does
- It slow, but it produces the best profiles
- The best predictor for many years was PSIPRED, an MLP that uses Psi-Blast profiles, getting close to 80% performance—now deep learning (CNNs, etc.) dominate

Summary

- Common machine learning tools have proved very useful in biological sequence analysis
- However, great care must be taken to ensure that the information fed into them is meaningful
- Often these have to be combined with other tools (such as profilers) to get really good performance

Summary

- Common machine learning tools have proved very useful in biological sequence analysis
- However, great care must be taken to ensure that the information fed into them is meaningful
- Often these have to be combined with other tools (such as profilers) to get really good performance

Summary

- Common machine learning tools have proved very useful in biological sequence analysis
- However, great care must be taken to ensure that the information fed into them is meaningful
- Often these have to be combined with other tools (such as profilers) to get really good performance